Spec-Driven Delivery

An automated system that clarifies intent, plans, builds, verifies and delivers — in any language, across many repositories, at a principal engineer's standard.

Project	enterprise-automation-core
Pipeline	intake → clarify → spec → plan → tasks → work → QA → deliver → harvest
Default persona	Principal AI + Full-Stack Engineer (25y)
Languages covered	19 toolchains, detected not assumed
Package	packages/future_agents/sdd/
Generated	2026-09-05

Every table and code listing in this document is generated from the source it describes. Regenerate with `python scripts/generate_handbook.py`.

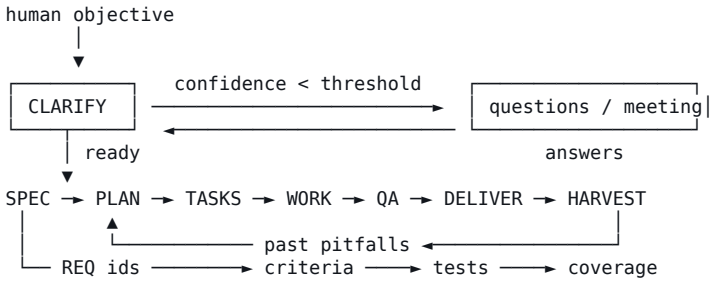
Contents

1 · What this system is

This is an automated delivery system. A human states an objective — in a meeting, a ticket, a chat message — and the system turns it into a specification, a technical plan, a dependency-ordered graph of test-first tasks, executed work, a QA verdict and a delivery record. When something is genuinely unclear it stops and asks; when the unknowns are too tangled for a form it asks for a meeting, arrives with an agenda, and resumes from the notes.

It is not a chatbot wrapped around a repository. Every stage is deterministic on its own: it derives its artifact from the upstream one by explicit rules. A language model, when configured, enriches free-text fields — it is an accelerator, never the source of truth. That is what makes runs reproducible, testable in CI, and safe to dry-run before anything touches a repository.

1.1 The loop



The clarification gate and the memory loop are the two feedback edges.

1.2 What it does that a plain agent does not

Property	How it is enforced	Where
Refuses to build on a guess	Intent is scored; blocking unknowns fail the gate closed	`clarify.py, constitution.py`
Asks a human at the right level	Ladder: auto-assume → async questions → meeting → blocked	`clarify.py`
Every requirement is traceable	REQ-001 → REQ-001-AC-001 → T-007 → QA check	`models.py, stages.py`
Tests exist before code	The implement task depends on its test task; parity is a gate	`stages.py, constitution.py`
Works in any language	19 detected toolchains supply install/test/lint/build commands	`languages.py`
Works at a stated seniority	Personas tune thresholds, add gates, inject heuristics as risks	`personas.py`
Spans repositories	Dependency waves, one merged question set for the whole program	`master.py`
Learns from failures	Cases are harvested; failures outrank successes in retrieval	`memory_hub.py`
Never silently assumes	Assumption ledger surfaces on the delivery record	`models.py, stages.py`

1.3 Reading this document

Chapters 2–6 are the core pipeline. Chapters 7–10 cover seniority, languages, structure and repository knowledge. Chapters 11–15 cover execution, QA, memory, routing and multi-repo orchestration. Chapters 16–18 are operational: configuration, the CLI and API, and the pattern

catalog. Chapters 19–21 cover extension, testing and honest limits.

2 · The philosophy: agents as compilers

A compiler does not guess what you meant. It parses source into an intermediate representation, and each pass is bounded by the one before it. Spec-driven delivery applies that shape to human intent: each artifact constrains the next, and a pass that cannot proceed fails loudly instead of inventing its input.

Artifact	Answers	Bounded by	Fails when
Objective	What does a human want?	nothing — the only un-derived input	never; it is raw intent
ClarificationResult	Do we understand it?	objective + prior answers	blocking unknowns remain
Spec	What and why?	the clarification outcome	a requirement has no acceptance criterion
Plan	How?	the spec's content hash	the spec changed underneath it (stale-plan)
TaskGraph	In what order?	the plan's content hash	a MUST requirement has no test task
WorkResult[]	What happened?	the task graph	a task fails; dependents block
QAReport	Did it work?	spec criteria + work results	coverage below the required bar
Delivery	Can we ship it?	the QA verdict + open questions	not accepted; assumptions surfaced
MemoryCase	What did we learn?	the whole run	never; a failed run is the most valuable case

2.1 Determinism first, model second

Every stage produces its artifact without calling a model. Requirements are extracted with modal-verb, imperative and transcript-attribution rules; components are grouped by domain keywords; the task graph is generated from the spec's shape. An engine, when configured, is asked to improve free-text fields only — a summary, an architecture paragraph. If the engine is missing, slow, or throws, the run continues and the structure is identical.

Why this matters
A pipeline whose structure depends on a model cannot be tested, cannot be replayed, and cannot be reasoned about when it goes wrong. Here, the model is the accelerator and the rules are the contract — so the entire system runs offline in CI, and the same objective produces the same graph every time.

2.2 Content hashes and staleness

Each artifact fingerprints its own semantic content, excluding provenance fields (ids, timestamps, confidence). A plan records the spec hash it was drawn from; if the spec is revised, the plan is stale and the constitution says so rather than letting the run continue on a superseded premise.

Fingerprinting excludes provenance, not content — `models.py :: Hashable`

```
class Hashable(BaseModel):
    """Artifact base that can fingerprint its own semantic content."""

    def content_hash(self) -> str:
        payload = self.model_dump(mode="json", exclude=self._hash_exclude())
        blob = json.dumps(payload, sort_keys=True, default=str)
        return hashlib.sha256(blob.encode()).hexdigest()[:16]
```

```
    def _hash_exclude(self) -> set[str]:
        # Timestamps and back-references are provenance, not content.
        return {"created_at", "updated_at", "id", "confidence"}
```

The stale-plan gate, among others — `constitution.py :: Constitution.check_plan`

```
    def check_plan(self, plan: Plan, spec: Optional[Spec] = None) -> list[Violation]:
        out: list[Violation] = []
        if spec is not None and plan.spec_hash != spec.content_hash():
            out.append(
                Violation(
                    rule="stale-plan",
                    detail="plan was drawn from a different spec revision — replan",
                    subject=plan.id,
                )
            )
        blob = " ".join(
            [plan.architecture, plan.test_strategy, *(c.responsibility for c in plan.components)]
        ).lower()
        for banned in self.banned_practices:
            if self._mentions(blob, banned):
                out.append(
                    Violation(
                        rule="banned-practice",
                        detail=banned,
                        subject=plan.id,
                    )
                )
        if len(plan.components) > self.max_component_fanout:
            out.append(
                Violation(
                    rule="component-fanout",
                    severity=Severity.WARN,
                    detail=f"{len(plan.components)} components exceeds {self.max_component_fanout}",
                    subject=plan.id,
                )
            )
        if not plan.test_strategy:
            out.append(Violation(rule="test-strategy-required", subject=plan.id))
        return out
```

3 · Architecture

3.1 Module map

Module	Responsibility	Key types
`models.py`	The IR artifacts and the run state	`Objective, Spec, Plan, TaskGraph, RunState`
`clarify.py`	Intent scoring, questions, meetings	`IntentClarifier, Signal, Question`
`constitution.py`	Executable governance gates	`Constitution, Violation, PatchDecision`
`config.py`	The rulebook loader	`SpecKitConfig, ConfigError`
`personas.py`	Seniority, heuristics, review gates	`Persona, Heuristic, ReviewGate`
`languages.py`	Toolchains and repo detection	`Toolchain, RepoProfile, detect_repo`
`scaffold.py`	Required repository structure	`RepoScaffolder, ScaffoldPlan`
`router.py`	Role/intent → engine, with fallback	`EngineRouter, Engine, NullEngine`
`memory_hub.py`	Case-based reasoning	`MemoryHub, MemoryCase, RetrievalReport`
`stages.py`	PM, Architect, Planner, Worker, QA, Delivery	`PMStage … DeliveryStage`
`pipeline.py`	The stage machine over one RunState	`DeliveryPipeline`
`master.py`	Many repos, one objective	`MasterOrchestrator, ProgramRun`
`handbook/`	This document, generated from the code	`build_handbook`

3.2 Control flow

```
DeliveryPipeline.start(objective)
  ├─ IntentClarifier.assess ............... score intent, produce questions
  │    └─ READY? no  → return RunState(stage=CLARIFY)   ← human answers here
  ├─ PMStage.draft ........................ Spec  + constitution.check_spec
  ├─ MemoryHub.retrieve ................... past pitfalls for this objective
  ├─ ArchitectStage.draft ................. Plan  + constitution.check_plan
  │    └─ persona.risks_for(spec) ......... 25 years, as risks
  ├─ TaskPlanner.build .................... TaskGraph + constitution.check_tasks
  │    ├─ test task ◄── implement task .... test-first by graph shape
  │    ├─ structure task (if repo incomplete)
  │    └─ persona.gate_tasks .............. security / migration / eval / perf …
  ├─ WorkerStage.execute .................. topological order, failures block deps
  ├─ QAStage.verify ....................... BDD + AAA, fences, coverage, verdict
  ├─ DeliveryStage.package ................ accepted? assumptions? residuals?
  └─ MemoryHub.harvest .................... one case, pitfalls first
```

3.3 Where a human enters

Exactly three places, and never in the middle of a stage: answering clarification questions, holding a clarification meeting, and reading the delivery record. The run state serialises to JSON at any point, so a run can wait days for a meeting and resume in a different process.

The stage machine, gate by gate — `pipeline.py :: DeliveryPipeline._build`

```
def _build(self, state: RunState) -> RunState:
    state.stage = Stage.SPEC
    spec = self.pm.draft(state.objective, state.clarification)
    violations = self.constitution.check_spec(spec)
    if self._blocking(violations):
        return self._block(state, Stage.SPEC, violations)
    state.spec = spec
    self._log(
        state,
        Stage.SPEC,
        f"{len(spec.requirements)} requirement(s), {len(spec.criteria())} criteria",
        spec_hash=spec.content_hash(),
    )
```

```
state.stage = Stage.PLAN
memory_report = self.memory.retrieve(f"{spec.title} {spec.summary}")
plan = self.architect.draft(spec, memory_report)
violations = self.constitution.check_plan(plan, spec)
if self._blocking(violations):
    return self._block(state, Stage.PLAN, violations)
state.plan = plan
self._log(
    state,
    Stage.PLAN,
    f"{len(plan.components)} component(s), {len(plan.historical_warnings)} warning(s)",
    cases=plan.memory_case_ids,
    placements=[p.summary() for p in plan.placements[:5]],
)

state.stage = Stage.TASKS
graph = self.planner.build(plan, spec)
violations = self.constitution.check_tasks(graph, spec)
if self._blocking(violations):
    return self._block(state, Stage.TASKS, violations)
state.tasks = graph
self._log(state, Stage.TASKS, f"{len(graph.tasks)} task(s) in the DAG")

state.stage = Stage.WORK
state.work_results = self.worker.execute(graph, spec)
done = sum(1 for r in state.work_results if r.status.value == "done")
self._log(state, Stage.WORK, f"{done}/{len(state.work_results)} task(s) completed")

state.stage = Stage.QA
state.qa = self.qa.verify(spec, graph, state.work_results)
self._log(
    state,
# … (see the source file for the rest)
```

4 · Intent clarification

This is the stage that separates a delivery system from a code generator. Most agent pipelines accept an underspecified objective and confidently build the wrong thing. Here, intent is scored before anything is built, and the system asks only what would change the outcome.

4.1 Detectors

Detector	Fires when	Cost	Blocking
vague terms	an unmeasurable adjective appears (fast, robust, TBD…)	0.14 each, max 3	no
missing metric	a change objective states no baseline or target	0.22	yes
missing acceptance	no observable outcome is stated	0.16	no — assumed
missing data source	data/report work names no system of record	0.20	yes
missing integration target	integration work names no counterparty	0.18	yes
dangling reference	the objective opens with it / this / they	0.25	yes
multi-objective	'and also', 'as well as' — two deliverables in one	0.12	no
escalation trigger	auth, payment, PII, PHI, migration, prod credential	0.20	yes
open-ended	ends in '?' or is shorter than six words	0.20–0.22	yes

The vague-term lexicon currently holds 27 entries: fast, faster, slow, scalable, robust, secure, simple, seamless, better, improve, improved, optimize, optimise, modern, …

4.2 The score

```
confidence = clamp(1 − Σ signal_weights, 0, 1) × structure_bonus

structure_bonus = 0.85
                + 0.05 if the objective carries constraints
                + 0.05 if it carries raw inputs (transcript, ticket)
                + 0.05 if it carries a deadline          (capped at 1.0)

an auto-assumed unknown costs half its weight — recorded, not free
```

Well-formed intake earns confidence back: an objective that arrives with constraints, a transcript and a date is a better-specified objective, and the score says so.

4.3 The escalation ladder

Rung	Condition	What happens
auto-assume	a low-risk unknown with a sensible default	an Assumption is recorded and surfaced at delivery
async questions	confidence between the two thresholds	a question set the human answers in their own time
meeting	confidence < meeting_threshold, or blocking unknowns survive max_rounds	a MeetingRequest with agenda, attendees, duration
blocked	a human stops the work, or the gate cannot be satisfied	the run stops with the reason recorded

The ladder, in code — `clarify.py :: IntentClarifier._decide`

```
def _decide(
    self,
    confidence: float,
```

```
            open_questions: list[Question],
            blocking_open: list[Question],
            round_number: int,
        ) -> tuple[ClarificationOutcome, str]:
            s = self.settings
            if confidence >= s.ready_threshold and not blocking_open:
                return ClarificationOutcome.READY, f"confidence {confidence} ≥ {s.ready_threshold}"
            if not open_questions:
                return ClarificationOutcome.READY, "no open questions remain"
            if confidence < s.meeting_threshold:
                return (
                    ClarificationOutcome.MEETING_REQUIRED,
                    f"confidence {confidence} < {s.meeting_threshold}: too tangled for async",
                )
            if round_number > s.max_rounds and blocking_open:
                return (
                    ClarificationOutcome.MEETING_REQUIRED,
                    f"{len(blocking_open)} blocking unknowns survived {s.max_rounds} async rounds",
                )
            return (
                ClarificationOutcome.ASYNC_QUESTIONS,
                f"{len(open_questions)} unknowns answerable without a meeting",
            )
```

4.4 Meetings

A meeting request is never a bare 'please clarify'. It carries the reason the system escalated, one agenda line per open question, the requester plus the configured attendees, and a duration. Closing it is a single call: the notes become objective context, the answers close the questions, and the run continues from where it stopped.

clarify.py :: IntentClarifier.record_meeting

```
        def record_meeting(
            self,
            objective: Objective,
            result: ClarificationResult,
            notes: str,
            answers: Optional[dict[str, str]] = None,
        ) -> ClarificationResult:
            """Close a meeting: notes become context, answers close the questions."""
            if result.meeting:
                result.meeting.status = MeetingStatus.HELD
                result.meeting.notes = notes
            objective.context = f"{objective.context}\n{notes}".strip()
            return self.apply_answers(objective, result, answers or {}, answered_by="meeting")
```

clarify.py :: IntentClarifier._meeting

```
        def _meeting(
            self,
            objective: Objective,
            result: ClarificationResult,
            questions: list[Question],
        ) -> MeetingRequest:
            s = self.settings
            attendees = list(s.meeting_attendees)
            if objective.submitted_by and objective.submitted_by not in attendees:
                attendees.insert(0, objective.submitted_by)
            return MeetingRequest(
                objective_id=objective.id,
                title=f"Clarify: {_short(objective.statement)}",
                reason=result.rationale,
                agenda=[q.text for q in questions],
                question_ids=[q.id for q in questions],
                required_attendees=attendees,
                duration_minutes=s.meeting_duration_minutes,
            )
```

4.5 Worked example

```
        >>> pipeline.start(Objective(statement="Make the dashboard faster", submitted_by="sam"))

        stage: clarify        intent: meeting_required (confidence 0.119)
          ·  'faster' is not measurable — what number or observable state counts as done?
```

```
  !  What is the current baseline and the target number?
  !  Which system of record supplies this data, and how fresh must it be?
  !  What problem does this solve, and for whom?
  assumed [medium] The requester verifies the outcome on the primary interface.

>>> pipeline.hold_meeting(state, 'Ops owns sign-off. Baseline 3.2s.', answers)

stage: done           intent: ready (confidence 0.782)
  QA PASS — 1/1 behaviours verified          delivery: ACCEPTED
```

One detector, end to end — `clarify.py :: _detect_missing_metric`

```
def _detect_missing_metric(objective: Objective, ctx: ClarifierContext) -> list[Signal]:
    wants_change = any(w in ctx.text for w in ("improve", "reduce", "increase", "optimi", "faster"))
    if _METRIC_HINT.search(ctx.raw):
        return []
    if not wants_change:
        return [
            Signal(
                topic=QuestionTopic.ACCEPTANCE,
                question="What is the success metric for this work?",
                why="delivery cannot be judged without one",
                weight=0.1,
                default_assumption="Success = all acceptance criteria verified by QA.",
            )
        ]
    return [
        Signal(
            topic=QuestionTopic.ACCEPTANCE,
            question="What is the current baseline and the target number?",
            why="a change objective without a baseline cannot be shown to have worked",
            weight=0.22,
            blocking=True,
        )
    ]
```

5 · The IR artifacts

5.1 Traceability identifiers

The single most valuable structural decision in the system: every requirement gets a stable id, every acceptance criterion hangs off that id, every task references the requirements and criteria it serves, and QA measures coverage over those ids. Coverage becomes computable rather than asserted.

```
REQ-002                    a requirement
└── REQ-002-AC-001         an acceptance criterion (Given / When / Then)
     ├── T-005  [test]     the test task that covers it
     ├── T-006  [code]     the implementation, depending on T-005
     └── QA check          verified = test task done AND code tasks done
```

5.2 Requirement and criterion

models.py :: Requirement

```
class Requirement(BaseModel):
    id: str  # REQ-001 — stable, referenced by tasks, tests and QA
    statement: str
    rationale: str = ""
    priority: Priority = Priority.MUST
    acceptance_criteria: list[AcceptanceCriterion] = Field(default_factory=list)
    source: str = ""
```

models.py :: AcceptanceCriterion

```
class AcceptanceCriterion(BaseModel):
    """Given/When/Then. The unit QA verifies and coverage is measured against."""

    id: str
    given: str
    when: str
    then: str

    def render(self) -> str:
        return f"Given {self.given}, when {self.when}, then {self.then}."
```

5.3 Spec

models.py :: Spec

```
class Spec(Hashable):
    """Functional intent — what and why. Deliberately free of tech-stack detail."""

    id: str = Field(default_factory=lambda: _nid("spec"))
    objective_id: str
    title: str
    summary: str = ""
    requirements: list[Requirement] = Field(default_factory=list)
    out_of_scope: list[str] = Field(default_factory=list)
    assumptions: list[Assumption] = Field(default_factory=list)
    open_questions: list[Question] = Field(default_factory=list)
    glossary: dict[str, str] = Field(default_factory=dict)
    success_metrics: list[str] = Field(default_factory=list)
    context_notes: list[str] = Field(default_factory=list)  # what the repo already does
    confidence: float = 0.0
    created_at: datetime = Field(default_factory=_now)

    def criteria(self) -> list[AcceptanceCriterion]:
        return [ac for r in self.requirements for ac in r.acceptance_criteria]

    def requirement(self, req_id: str) -> Optional[Requirement]:
        return next((r for r in self.requirements if r.id == req_id), None)
```

5.4 Plan

models.py :: Plan

```
class Plan(Hashable):
    """Technical blueprint — how. Bound by the spec hash it was drawn from."""

    id: str = Field(default_factory=lambda: _nid("plan"))
    spec_id: str
    spec_hash: str
    architecture: str = ""
    runtime_stack: str = ""
    components: list[Component] = Field(default_factory=list)
    data_contracts: list[str] = Field(default_factory=list)
    test_strategy: str = ""
    risks: list[Risk] = Field(default_factory=list)
    historical_warnings: list[str] = Field(default_factory=list)
    memory_case_ids: list[str] = Field(default_factory=list)
    placements: list[PlacementDecision] = Field(default_factory=list)
    reuse_candidates: list[RepoMatch] = Field(default_factory=list)
    confidence: float = 0.0
    created_at: datetime = Field(default_factory=_now)

    def placement_for(self, requirement_id: str) -> Optional[PlacementDecision]:
        return next((p for p in self.placements if p.requirement_id == requirement_id), None)
```

5.5 The task graph

Kahn's algorithm with a stable queue: the same graph always produces the same order, which matters when a run is replayed. A cycle raises rather than silently dropping tasks — a dropped task is a requirement that quietly never ships.

`models.py :: TaskGraph.topological_order`

```
def topological_order(self) -> list[TaskUnit]:
    """Kahn's algorithm. Raises CycleError rather than dropping tasks."""
    indegree = {t.id: 0 for t in self.tasks}
    dependents: dict[str, list[str]] = {t.id: [] for t in self.tasks}
    for task in self.tasks:
        for dep in task.depends_on:
            if dep not in indegree:
                raise CycleError(f"{task.id} depends on unknown task {dep}")
            indegree[task.id] += 1
            dependents[dep].append(task.id)

    # Ordered queue keeps the output stable across runs.
    queue = sorted([tid for tid, deg in indegree.items() if deg == 0])
    ordered: list[TaskUnit] = []
    while queue:
        tid = queue.pop(0)
        task = self.by_id(tid)
        if task is not None:
            ordered.append(task)
        for child in dependents[tid]:
            indegree[child] -= 1
            if indegree[child] == 0:
                queue.append(child)
        queue.sort()

    if len(ordered) != len(self.tasks):
        unresolved = sorted(set(indegree) - {t.id for t in ordered})
        raise CycleError(f"cycle in task graph: {', '.join(unresolved)}")
    return ordered
```

5.6 Run state

The pipeline holds no state of its own. Everything lives in the RunState, which is a Pydantic model and therefore JSON on demand — the reason a run can pause for a meeting and resume in another process, another day, another machine.

`models.py :: RunState`

```
class RunState(BaseModel):
    """The whole run, resumable from JSON — the pipeline holds no other state."""

    id: str = Field(default_factory=lambda: _nid("run"))
    objective: Objective
    stage: Stage = Stage.INTAKE
```

```
    clarification: Optional[ClarificationResult] = None
    spec: Optional[Spec] = None
    plan: Optional[Plan] = None
    tasks: Optional[TaskGraph] = None
    work_results: list[WorkResult] = Field(default_factory=list)
    qa: Optional[QAReport] = None
    delivery: Optional[Delivery] = None
    case_id: Optional[str] = None
    events: list[PipelineEvent] = Field(default_factory=list)
    clarification_rounds: int = 0
    updated_at: datetime = Field(default_factory=_now)

    def log(self, stage: Stage, message: str, **data: Any) -> None:
        self.events.append(PipelineEvent(stage=stage, message=message, data=data))
        self.updated_at = _now()

    @property
    def awaiting_human(self) -> bool:
        return self.stage is Stage.CLARIFY and self.clarification is not None

    def pending_questions(self) -> list[Question]:
        if not self.clarification:
            return []
        return [q for q in self.clarification.questions if not q.answered]
```

6 · The constitution and its gates

The constitution is data, not prose, so a gate can evaluate it. The markdown form that agents read is rendered from the same object, which is why the two cannot drift apart. Errors fail a stage closed; warnings are recorded and the run continues.

6.1 Rules

Rule	Severity	What it catches
`acceptance-criteria-required`	error	a requirement with nothing to verify
`no-blocking-unknowns`	error	building on an unanswered blocking question
`spec-purity`	warn	a functional spec naming the tech stack
`stale-plan`	error	a plan drawn from a superseded spec revision
`banned-practice`	error	a plan brushing a governance ban
`test-strategy-required`	error	a plan with no stated test approach
`component-fanout`	warn	a design fragmented past the configured limit
`test-parity`	error	a MUST requirement with no test task
`untraceable-task`	warn	a code task that serves no requirement

6.2 The current governance set

Setting	Value
runtime_stack	Python 3.11+, FastAPI, Pydantic v2
banned_practices	No direct database connections from API route handlers. No secrets in code — environment variables or a secrets manager only. No exact version pins without written justification.
security_boundaries	Agents never hold long-lived production credentials. Outbound calls carry no user financial or personal context.
escalation_triggers	auth, payment, pii, phi, migration, production credential
stack_terms (spec purity)	postgres, kubernetes, react, kafka, redis, lambda
enforce_test_parity	True
enforce_spec_purity	True

Read live from data/config/spec_kit/spec-kit-enterprise.yaml at generation time.

6.3 The spec gate

`constitution.py :: Constitution.check_spec`

```
        def check_spec(self, spec: Spec) -> list[Violation]:
            out: list[Violation] = []
            for req in spec.requirements:
                if not req.acceptance_criteria:
                    out.append(
                        Violation(
                            rule="acceptance-criteria-required",
                            detail=f"{req.id} has no Given/When/Then criteria",
                            subject=req.id,
                        )
                    )
                if self.enforce_spec_purity:
                    hit = self._stack_leak(req.statement)
                    if hit:
```

```
                    out.append(
                        Violation(
                            rule="spec-purity",
                            severity=Severity.WARN,
                            detail=f"{req.id} names implementation detail '{hit}'",
                            subject=req.id,
                        )
                    )
        for question in spec.open_questions:
            if question.blocking and not question.answered:
                out.append(
                    Violation(
                        rule="no-blocking-unknowns",
                        detail=f"unanswered blocking question: {question.text}",
                        subject=question.id,
                    )
                )
        return out
```

6.4 The task gate — test parity

`constitution.py :: Constitution.check_tasks`

```
    def check_tasks(self, graph: TaskGraph, spec: Spec) -> list[Violation]:
        out: list[Violation] = []
        if not self.enforce_test_parity:
            return out
        tested = {
            rid
            for task in graph.tasks
            if task.kind is TaskKind.TEST
            for rid in task.requirement_ids
        }
        for req in spec.requirements:
            if req.priority.value == "must" and req.id not in tested:
                out.append(
                    Violation(
                        rule="test-parity",
                        detail=f"{req.id} has no test task",
                        subject=req.id,
                    )
                )
        for task in graph.tasks:
            if task.kind is TaskKind.CODE and not task.requirement_ids:
                out.append(
                    Violation(
                        rule="untraceable-task",
                        severity=Severity.WARN,
                        detail=f"{task.id} traces to no requirement",
                        subject=task.id,
                    )
                )
        return out
```

6.5 Banned-practice matching

Bans are written as sentences ('No direct database connections from API route handlers.'), but they must fire on prose that says the same thing differently. The matcher extracts content words and requires a supermajority to hit, which catches paraphrase without firing on every mention of the word 'database'.

`constitution.py :: Constitution._mentions`

```
    @staticmethod
    def _mentions(haystack: str, needle: str) -> bool:
        # Banned practices are written as sentences; match on their content words
        # so "No direct database connections from API route handlers." still
        # fires on "database connection in the route handler".
        words = [w for w in re.findall(r"[a-z]{4,}", needle.lower()) if w not in _FILLER]
        if not words:
            return needle.lower() in haystack
        if len(words) <= 2:
            return all(w in haystack for w in words)
        hits = sum(1 for w in words if w in haystack)
        return hits >= max(2, int(len(words) * 0.6))
```

6.6 The CI/CD diff gate

Agents rewrite pipelines. The golden template is the approved shape; a proposed change may add steps but may not remove a topology line — a job, a needs edge, a runner, a uses, a steps or strategy block. The gate is a structural diff, not a prompt instruction, so it holds regardless of which engine produced the change.

constitution.py :: Constitution.diff_gate

```
        def diff_gate(self, golden: str, proposed: str) -> "PatchDecision":
            """Allow additive patches to a golden pipeline; reject topology rewrites.

            `golden` is the approved template, `proposed` what an agent produced.
            Removing or reordering existing structural lines is a rewrite.
            """
            golden_lines = golden.splitlines()
            proposed_lines = proposed.splitlines()
            matcher = difflib.SequenceMatcher(None, golden_lines, proposed_lines)

            added: list[str] = []
            removed: list[str] = []
            for tag, i1, i2, j1, j2 in matcher.get_opcodes():
                if tag in ("delete", "replace"):
                    removed.extend(golden_lines[i1:i2])
                if tag in ("insert", "replace"):
                    added.extend(proposed_lines[j1:j2])

            structural = [line for line in removed if _is_topology(line)]
            if structural:
                return PatchDecision(
                    allowed=False,
                    reason="proposed change removes golden pipeline topology",
                    removed_topology=structural,
                    added_lines=added,
                )
            return PatchDecision(
                allowed=True,
                reason="additive patch",
                removed_topology=[],
                added_lines=added,
                removed_lines=removed,
            )

        $ python scripts/spec_kit.py diff-gate --proposed .github/workflows/ci.yml
        {'allowed': False, 'added': 4, 'removed': 9,
         'reason': 'proposed change removes golden pipeline topology'}
          removed topology: jobs:
          removed topology: needs: lint
```

7 · Personas: working at 25 years of experience

A persona is not a tone of voice. It is a set of behavioural changes: how much confidence the system demands before it starts building, which review gates are mandatory in the task graph, what coverage it will accept, and which hard-won rules enter the plan as explicit risks. The default is the principal hybrid — one engineer who has carried both the model pager and the request-path pager.

7.1 The catalog

Persona	Yrs	Ready	Coverage	Heuristics	Mandatory gates
Principal AI + Full-Stack Engineer `principal_hybrid`	25	0.8	1.0	17	Security review Data migration and rollback review Model evaluation gate Performance budget check Observability and rollback readiness ADR and runbook
Principal AI Engineer `principal_ai_engineer`	25	0.8	1.0	9	Model evaluation gate Security review Observability and rollback readiness ADR and runbook
Principal Full-Stack Engineer `principal_fullstack`	25	0.78	1.0	11	Security review Data migration and rollback review Observability and rollback readiness ADR and runbook
Staff Platform / SRE `staff_platform`	18	0.75	—	5	Observability and rollback readiness Security review ADR and runbook
Senior Engineer (pragmatic) `pragmatic`	8	0.7	0.8	3	ADR and runbook

7.2 What a persona actually changes

Effect	Mechanism	Consequence
Interrogation depth	ready_threshold / meeting_threshold	a principal asks more before building
Coverage bar	qa.required_coverage	1.0 for principals, 0.8 for the pragmatic profile
Mandatory review	gate_tasks() appends REVIEW units	security, migration, eval, perf, observability, ADR
Design constraints	risks_for() appends plan risks	experience becomes a constraint, not advice
Engine choice	engine_overrides per role	a harder role gets a stronger model

`personas.py :: Persona.apply_to_config`

```
        def apply_to_config(self, config: SpecKitConfig) -> SpecKitConfig:
            """Return a copy of the rulebook as this persona would set it."""
            tuned = config.model_copy(deep=True)
            if self.ready_threshold is not None:
                tuned.clarification.ready_threshold = self.ready_threshold
            if self.meeting_threshold is not None:
                tuned.clarification.meeting_threshold = self.meeting_threshold
            if self.max_clarification_rounds is not None:
                tuned.clarification.max_rounds = self.max_clarification_rounds
            if self.required_coverage is not None:
                tuned.qa.required_coverage = self.required_coverage
```

```
            for role, engine in self.engine_overrides.items():
                if role in tuned.agents.roles:
                    tuned.agents.roles[role].engine = engine
            return tuned
```

personas.py :: Persona.gate_tasks

```
        def gate_tasks(self, spec: Spec, depends_on: list[str], start_index: int) -> list[TaskUnit]:
            """Mandatory review units this persona adds before delivery."""
            blob = " ".join(
                [spec.title, spec.summary, *(r.statement for r in spec.requirements)]
            ).lower()
            tasks: list[TaskUnit] = []
            index = start_index
            for gate in self.gates:
                if not gate.matches(blob):
                    continue
                index += 1
                tasks.append(
                    TaskUnit(
                        id=f"T-{index:03d}",
                        title=gate.name,
                        description=gate.description,
                        kind=TaskKind.REVIEW,
                        requirement_ids=[r.id for r in spec.requirements],
                        depends_on=list(depends_on),
                        engine="",
                    )
                )
            return tasks
```

7.3 The heuristics of the default persona

17 rules, each with the keywords that make it relevant. A heuristic that fires becomes a Risk on the plan with `source=persona:principal_hybrid`, so a reviewer can see exactly why it is there.

Heuristic	Fires on	Severity
Define the eval set and the acceptance bar before touching the model or prompt — 'it looks better' is not a result.	model, prompt, llm, embedding, rag, agent, inference, ml	high
Pin the model id and record it with the output; a silently upgraded model is an unversioned dependency.	model, llm, prompt, inference, agent	high
Cap context and cost per request, and log both — token spend is a production resource like memory.	llm, prompt, agent, rag, context, token	medium
Treat retrieved documents and tool output as untrusted input: they can carry instructions.	rag, retrieval, agent, tool, scrape, webhook, mcp	high
Non-determinism needs a seed, a snapshot, or a tolerance — never an exact-match assertion on generated text.	model, llm, generate, prompt, agent	medium
Measure the baseline before optimising; a speedup with no baseline is a story.	performance, latency, optimi, faster, throughput, scale	medium
Every schema change ships with a migration and a tested rollback path.	schema, migration, database, table, column, index	high
An API change is a contract change: version it, or keep the old shape working.	api, endpoint, contract, payload, response, graphql, grpc	high
Validate at the boundary, encode on output — never trust a client-side check.	input, form, api, upload, user, request, endpoint	high
Anything crossing the network needs a timeout, a retry policy with backoff, and an idempotency key if it writes.	http, api, request, webhook, integration, queue, sync	medium

Heuristic	Fires on	Severity
Ship it observable: a log line, a metric and an alert threshold, decided before release, not after the incident.	always	medium
Feature-flag anything that changes behaviour for existing users; the rollback is the flag, not a redeploy.	release, rollout, behaviour, behavior, ui, migration, flow	medium
Cache with an explicit key, TTL and invalidation trigger — an unowned cache becomes a stale-data bug.	cache, redis, memoi, performance, latency	medium
N+1 queries and unbounded result sets are the two defects that reach production most often: paginate and check the query count.	query, list, database, orm, report, export, search	medium
Write the runbook entry with the change: what breaks, how you see it, how you undo it.	always	medium
Record the decision as an ADR when the choice would be expensive to reverse.	architecture, framework, database, protocol, vendor, platform	medium
Secrets come from the environment or a manager, are never logged, and rotate without a code change.	secret, key, token, credential, auth, password	high

7.4 Selection

```
python scripts/spec_kit.py --persona principal_ai_engineer run --statement '…'
python scripts/spec_kit.py --persona pragmatic run --statement '…'   # internal tool

DeliveryPipeline(config, persona=get_persona('principal_fullstack'))
MasterOrchestrator(config, persona=PRINCIPAL_HYBRID)      # per-repo override too
```

Unknown persona ids do not raise
get_persona() falls back to the principal hybrid. A typo in a config file should degrade to the strictest sensible default, never crash a delivery run or silently drop every gate.

8 · Any language: the toolchain matrix

A repository is profiled from evidence — its manifests and its file extensions — never assumed. Each language carries the commands a contributor actually runs, the layout its ecosystem expects, and the dependency-pinning policy the guardrails enforce for it. Nothing in the pipeline hard-codes 'pytest': every stage asks the toolchain.

8.1 The 19 toolchains

Language	Detected by	Test	Lint	Pin
Python	`pyproject.toml, setup.py`	`pytest -q`	`ruff check .`	`~=`
TypeScript	`tsconfig.json`	`npm test`	`npm run lint`	`^`
JavaScript	`package.json`	`npm test`	`npx eslint .`	`^`
Go	`go.mod`	`go test ./...`	`golangci-lint run`	exact
Rust	`Cargo.toml`	`cargo test`	`cargo clippy -- -D warnings`	`^`
Java	`pom.xml, build.gradle`	`mvn -B test`	`mvn -B checkstyle:check`	exact
Kotlin	`build.gradle.kts, settings.gradle.kts`	`./gradlew test`	`./gradlew ktlintCheck`	exact
C#/.NET	`*.csproj, *.sln`	`dotnet test`	`dotnet format --verify-no-changes`	exact
Ruby	`Gemfile, *.gemspec`	`bundle exec rspec`	`bundle exec rubocop`	`~>`
PHP	`composer.json`	`./vendor/bin/phpunit`	`./vendor/bin/phpcs`	`^`
Swift	`Package.swift`	`swift test`	`swiftlint`	`from:`
C/C++	`CMakeLists.txt, meson.build`	`ctest --test-dir build --output-on-failure`	`clang-tidy -p build`	exact
Scala	`build.sbt`	`sbt test`	`sbt scalafixAll --check`	exact
Elixir	`mix.exs`	`mix test`	`mix credo --strict`	`~>`
Dart/Flutter	`pubspec.yaml`	`dart test`	`dart analyze`	`^`
R	`DESCRIPTION, renv.lock`	`Rscript -e 'testthat::test_dir("tests")'`	`Rscript -e 'lintr::lint_dir()'`	`renv.lock`
SQL/dbt	`dbt_project.yml`	`dbt test`	`sqlfluff lint .`	`range`
Terraform/IaC	`main.tf, versions.tf`	`terraform validate`	`tflint`	exact
Shell	`.sh, .bash`	`bats tests`	`shellcheck $(git ls-files '*.sh')`	`n/a`

8.2 Dependency policy per ecosystem

The guardrails rule 'no exact pins' is correct for application dependencies in Python and npm, and wrong for Go, Maven and Terraform providers, where exact pinning is the ecosystem's design. The matrix encodes the difference so the rule is applied with judgement rather than uniformly.

Language	Dependency policy
Python	compatible-release ranges; exact pins only for build tooling
TypeScript	caret ranges in package.json; lockfile committed

Language	Dependency policy
JavaScript	caret ranges in package.json; lockfile committed
Go	go.mod pins exactly by design; keep go.sum committed
Rust	caret by default in Cargo.toml; Cargo.lock committed for binaries
Java	exact versions are correct in pom.xml; never LATEST
Kotlin	version catalog (libs.versions.toml) is the single place versions live
C#/.NET	PackageReference pins exactly; centralise in Directory.Packages.props
Ruby	pessimistic constraint (~>) in the Gemfile; Gemfile.lock committed
PHP	caret ranges in composer.json; composer.lock committed
Swift	`.upToNextMajor(from:)` in Package.swift; Package.resolved committed
C/C++	pin dependency versions in conanfile/vcpkg.json; commit the lockfile
Scala	exact versions in build.sbt; centralise in Dependencies.scala
Elixir	`~>` in mix.exs; mix.lock committed
Dart/Flutter	caret ranges in pubspec.yaml; pubspec.lock committed for apps
R	renv.lock is the pin; commit it
SQL/dbt	version ranges in packages.yml; package-lock.yml committed
Terraform/IaC	pin provider versions exactly in versions.tf; commit .terraform.lock.hcl
Shell	pin tool versions in the container image, not in the script

8.3 Detection

A manifest is stronger evidence than a pile of files: a repo with one `go.mod` and four hundred JSON fixtures is a Go repo. Manifests score 50, files score 1, and TypeScript never loses to JavaScript merely because both carry a package.json.

`repos/languages.py :: detect_repo`

```
        def detect_repo(root: str | Path, max_files: int = 20000) -> RepoProfile:
            """Profile a repository from its manifests and file extensions."""
            root_path = Path(root)
            ext_counts: Counter[str] = Counter()
            manifests: dict[str, list[str]] = {}
            file_total = 0
            has_ci = (root_path / ".github" / "workflows").is_dir() or (
                root_path / ".gitlab-ci.yml"
            ).is_file()
            has_container = any(
                (root_path / name).is_file() for name in ("Dockerfile", "Containerfile", "compose.yaml")
            )
            workspace_markers = 0

            for path in _walk(root_path, max_files):
                file_total += 1
                name = path.name
                suffix = path.suffix
                if suffix:
                    ext_counts[suffix] += 1
                for chain in TOOLCHAINS:
                    for manifest in chain.manifests:
                        if _manifest_match(name, manifest):
                            manifests.setdefault(chain.language, []).append(
                                str(path.relative_to(root_path))
                            )
                            if path.parent != root_path:
                                workspace_markers += 1

            signals: list[LanguageSignal] = []
```

```
    for chain in TOOLCHAINS:
        files = sum(ext_counts.get(ext, 0) for ext in chain.extensions)
        found = manifests.get(chain.language, [])
        if not files and not found:
            continue
        # A manifest is stronger evidence than a pile of files: a repo with one
        # go.mod and 400 .json fixtures is a Go repo.
        score = files + 50.0 * len(found)
        signals.append(
            LanguageSignal(language=chain.language, files=files, manifests=found[:5], score=score)
        )

    signals.sort(key=lambda s: s.score, reverse=True)
    primary = signals[0].language if signals else "unknown"
    # TypeScript repos always carry package.json; don't let JS outrank TS.
    if primary == "javascript" and any(s.language == "typescript" for s in signals):
    # … (see the source file for the rest)
```

8.4 A polyglot repository

Secondary languages are kept, not discarded. The scaffolder emits an extra CI workflow per significant secondary toolchain, and the profile exposes every detected language so a plan can name the right commands for the part of the repo it touches.

```
$ python scripts/spec_kit.py detect --path .
.: python — detected: python(184), javascript(2), shell(2), sql(1)
  monorepo: True  ci: True  tests: True
  toolchain: Python
    install   pip install -e '.[dev]'
    format    ruff format .
    lint      ruff check .
    test      pytest -q
  dependency policy: compatible-release ranges; exact pins only for build tooling
  missing structure: docs/architecture.md, docs/runbook.md, docs/adr/0001-…md
```

8.5 Adding a language

One `Toolchain` entry. No other module changes — detection, scaffolding, CI generation, the plan's test strategy and the task descriptions all read from it.

```
Toolchain(
    language="zig",
    display_name="Zig",
    manifests=("build.zig",),
    extensions=(".zig",),
    package_manager="zig",
    install="zig build --fetch",
    test="zig build test",
    format="zig fmt .",
    build="zig build -Doptimize=ReleaseSafe",
    pin_style="exact",
    pin_rule="dependencies pinned by hash in build.zig.zon",
    layout=_common("src", "tests"),
    manifest_file="build.zig",
    manifest_template='const std = @import("std");\n',
)
```

9 · Repository structure

Every repository the system touches must have the same governance surface, whatever language it is written in. The scaffolder computes what is missing without touching disk, and writes only the gaps — it never overwrites, and it never creates a forbidden file.

9.1 The universal surface

Entry	Why it is required
`README.md`	what this is and the exact commands to run it
`.gitignore`	secrets and build output never reach the remote
`.env.example`	every variable the app reads, with REPLACE_ME placeholders
`docs/architecture.md`	the shape of the system and its trust boundaries
`docs/runbook.md`	what breaks, how you see it, how you undo it
`docs/adr/0001-…md`	decisions that are expensive to reverse
`.github/workflows/ci.yml`	lint → test → guardrails, in the golden topology
`<language manifest>`	a correct starter manifest when the repo has none
`<source> / <tests>`	the layout that language's ecosystem expects

9.2 Per-language layout

Language	Source	Tests	Extra
Python	`src`	`tests`	`tests/unit, tests/integration`
TypeScript	`src`	`tests`	`src/routes, src/services, tsconfig.json`
JavaScript	`src`	`tests`	—
Go	`internal`	—	`cmd`
Rust	`src`	`tests`	—
Java	`src/main/java`	`src/test/java`	`src/main/resources`
Kotlin	`src/main/kotlin`	`src/test/kotlin`	—
C#/.NET	`src`	`tests`	—
Ruby	`lib`	`spec`	—
PHP	`src`	`tests`	—
Swift	`Sources`	`Tests`	—
C/C++	`src`	`tests`	`include`
Scala	`src/main/scala`	`src/test/scala`	—
Elixir	`lib`	`test`	—
Dart/Flutter	`lib`	`test`	—
R	`R`	`tests`	—
SQL/dbt	`models/staging`	`tests`	`models/marts, docs/data-dictionary.md`
Terraform/IaC	`modules`	—	`environments/dev, environments/prod, versions.tf`

Language	Source	Tests	Extra
Shell	scripts	tests	–

9.3 Never created

The scaffolder refuses these regardless of what a plan or an engine asks for: .env, credentials.json, secrets.json, id_rsa, id_ed25519, terraform.tfvars. A `.env.example` is always written; a `.env` never is.

9.4 The generated CI workflow

Built from the detected toolchain's own commands, in the golden topology — three jobs wired with needs, so the diff gate can later protect it from being flattened.

repos/scaffold.py :: _ci_workflow

```
        def _ci_workflow(name: str, chain: Toolchain) -> str:
            """Golden topology: lint → test → guardrails, jobs wired with `needs`."""
            install = f"      - run: {chain.install}\n" if chain.install else ""
            lint = chain.lint or "echo 'no linter configured'"
            test = chain.test or "echo 'no test command configured'"
            return f"""name: {name} ci

        on:
          push:
            branches: [main]
          pull_request:

        jobs:
          lint:
            runs-on: {chain.ci_image}
            steps:
              - uses: actions/checkout@v4
        {install}      - run: {lint}

          test:
            needs: lint
            runs-on: {chain.ci_image}
            steps:
              - uses: actions/checkout@v4
        {install}      - run: {test}

          guardrails:
            needs: test
            runs-on: {chain.ci_image}
            steps:
              - uses: actions/checkout@v4
              - run: echo 'wire the guardrails engine here'
        """
```

9.5 Planning and applying

repos/scaffold.py :: RepoScaffolder.plan

```
        def plan(
            self,
            root: str | Path,
            profile: Optional[RepoProfile] = None,
            language: Optional[str] = None,
            name: str = "",
            description: str = "",
        ) -> ScaffoldPlan:
            root_path = Path(root)
            if language:
                chain = toolchain_for(language) or GENERIC
                detected = profile or RepoProfile(root=str(root_path), primary_language=chain.language)
            else:
                detected = profile or detect_repo(root_path)
                chain = detected.toolchain()

            repo_name = name or root_path.resolve().name
```

```
    actions: list[ScaffoldAction] = []
    seen: set[str] = set()

    def add(
        path: str, kind: str, purpose: str, content: str = "", required: bool = True
    ) -> None:
        if path in seen or Path(path).name in FORBIDDEN:
            return
        seen.add(path)
        exists = _satisfied(root_path, path, purpose)
        actions.append(
            ScaffoldAction(
                path=path,
                kind=kind,
                action="exists" if exists else "create",
                purpose=purpose,
                required=required,
                content="" if exists else content,
            )
        )

    for entry in chain.layout:
        content = (
            entry.template.format(
                name=repo_name,
                description=description or f"{chain.display_name} service.",
                install=chain.install or "# install",
                test=chain.test or "# test",
            )
    # … (see the source file for the rest)
```

9.6 Monorepos

A monorepo keeps its source under `packages/` or `apps/`, not `src/`. The validator accepts any conventional root rather than littering an established repository with an empty directory it does not want.

repos/scaffold.py :: _satisfied

```
def _satisfied(root: Path, path: str, purpose: str) -> bool:
    if (root / path).exists():
        return True
    aliases = _ALIASES.get(purpose, ())
    if not aliases or "/" in path:
        return False
    return any((root / alias).is_dir() for alias in aliases)
```

9.7 As a delivery gate

When a pipeline is given a repo root, the planner asks the scaffolder what is missing and, if anything is, adds an INFRA task naming the gaps — the structure work becomes part of the delivery instead of a lint failure three weeks later.

```
$ python scripts/spec_kit.py scaffold --path ../new-service --language go --write
go: 10 to create, 0 already present
  + go.mod                    go modules manifest
  + cmd                       entrypoints
  + internal                  source
  + docs                      architecture notes and runbooks
  + README.md                 what this is, how to run it
  + .env.example              every env var with REPLACE_ME placeholders
  + .gitignore                never commit secrets or build output
  + docs/architecture.md      the shape of the system
  + docs/runbook.md           what breaks, how you see it, how you undo it
  + .github/workflows/ci.yml  lint → test → guardrails, the golden topology
```

10 · Repository knowledge: where a change may and may not go

A spec that does not know the repository produces plans nobody can act on: a component with no home, a duplicate of something that already exists, a file written into a generated directory. This stage indexes the repository and answers three questions before the plan is drawn — what already exists, where this change belongs, and where it must never land.

10.1 What gets indexed

Source	What is taken	Used for
Python files	module docstring, classes, functions, method names (AST)	reuse candidates, duplicate risk
Other code	symbol names by language-specific pattern, leading comment	the same, in any language
Docs	first heading	context, not placement
Directories	kind (source / tests / docs / data / generated), purpose, file count	target and test paths, fences
`AGENTS.md`, `CLAUDE.md`, `CONTRIBUTING.md`	'where does a new X go' tables, prohibitions	the decisive placement rules
Toolchain	the ecosystem's layout and test glob	fallback when nothing else applies

On this repository: 609 files, 1955 symbols, 17 placement rules from AGENTS.md, CLAUDE.md, docs/ARCHITECTURE.md, README.md, built in well under a second.

10.2 Retrieval

TF-IDF over path segments, symbol names and docstrings, with symmetric suffix stripping so 'invoices' finds `invoice_agent.py`. Lexical, not semantic — it builds in a second, runs offline, and returns the same answer twice. `RepoIndex.search` is the single seam an embedding store would replace.

`knowledge/index.py :: RepoIndex.search`

```
        def search(
            self,
            query: str,
            limit: int = 6,
            kinds: Iterable[str] = (),
            require_name_overlap: int = 0,
        ) -> list[RepoMatch]:
            """TF-IDF over path segments, symbol names and docs.

            `require_name_overlap` demands that many query words appear in the *path
            or symbol name*, not merely in prose. Docs alone match almost anything;
            a name match is what makes "this already exists" believable.
            """
            terms = _tokens(query)
            if not terms:
                return []
            wanted = set(kinds)
            scores: dict[str, float] = defaultdict(float)
            for term in terms:
                idf = self._idf.get(term)
                if idf is None:
                    continue
                for path, count in self._postings.get(term, {}).items():
                    scores[path] += (1 + math.log(count)) * idf

            term_set = set(terms)
```

```
    matches: list[RepoMatch] = []
    for path, score in scores.items():
        note = self.files[path]
        if wanted and note.kind not in wanted:
            continue
        symbol = _best_symbol(note, terms)
        if require_name_overlap and len(term_set & _name_tokens(note)) < require_name_overlap:
            continue
        matches.append(
            RepoMatch(
                path=path,
                symbol=symbol.name if symbol else "",
                kind=symbol.kind if symbol else note.kind,
                score=round(score / (self._norm.get(path) or 1.0), 4),
                excerpt=(symbol.doc if symbol and symbol.doc else note.doc)[:200],
                reason="name and documentation overlap",
            )
        )
    matches.sort(key=lambda m: m.score, reverse=True)
    return matches[:limit]
```

10.3 Believable matches only

Prose overlap alone matches almost anything, and a false 'this already exists' teaches people to ignore the warning that matters. A duplicate-risk claim therefore requires two query words in the path or a symbol name, not merely in a docstring.

`knowledge/__init__.py :: RepoKnowledge.duplicate_risk`

```
    def duplicate_risk(self, requirement: Requirement, threshold: float = 0.35) -> list[RepoMatch]:
        """Existing code close enough that the requirement may already be met.

        Deliberately strict: two query words must appear in the path or symbol
        name. A weak prose match reported as a duplicate is noise, and noise here
        trains people to ignore the warning that matters.
        """
        return [
            match
            for match in self.index.search(
                requirement.statement, limit=2, kinds=("code",), require_name_overlap=2
            )
            if match.score >= threshold
        ]
```

10.4 The repository's own rules win

Most teams have already written down where things go. Parsing that table beats inventing a policy, and it lets every decision cite the file it came from.

The repo says	Destination	From
A new agent type	`packages/future_agents/agents/_agent.py`	AGENTS.md
An agentic pattern	`packages/future_agents/patterns/`	AGENTS.md
A spec-driven delivery stage or gate	`packages/future_agents/sdd/`	AGENTS.md
Support for another language	`packages/future_agents/sdd/repos/languages.py`	AGENTS.md
A repo-knowledge rule or detector	`packages/future_agents/sdd/knowledge/`	AGENTS.md
A seniority profile	`packages/future_agents/sdd/personas.py`	AGENTS.md
A scheduled worker	`packages/future_agents/workers/`	AGENTS.md
A guardrail rule	`packages/guardrails/`	AGENTS.md

Reading a 'where does it go' table — `knowledge/conventions.py :: _parse_tables`

```
    def _parse_tables(text: str, source: str) -> list[PlacementRule]:
        """Read markdown tables whose header says 'goes in' / 'where'."""
        rules: list[PlacementRule] = []
        rows: list[list[str]] = []
        header: list[str] = []
```

```
    for line in text.splitlines():
        stripped = line.strip()
        if not stripped.startswith("|"):
            header, rows = [], []
            continue
        cells = [c.strip() for c in stripped.strip("|").split("|")]
        if set("".join(cells)) <= set("-: "):
            continue  # the separator row
        if not header:
            header = [c.lower() for c in cells]
            continue
        rows.append(cells)

        where_index = _column(header, _WHERE_HEADERS)
        subject_index = _column(header, _SUBJECT_HEADERS)
        if where_index is None or subject_index is None or where_index >= len(cells):
            continue
        subject = _clean(cells[subject_index])
        destination = _first_path(cells[where_index]) or _clean(cells[where_index])
        if not subject or not destination:
            continue
        note = " ".join(
            _clean(cells[i]) for i in range(len(cells)) if i not in {where_index, subject_index}
        )
        rules.append(
            PlacementRule(
                subject=subject,
                destination=destination,
                note=note[:200],
                source=source,
                keywords=_tokens(subject),
            )
        )
    return rules
```

10.5 Where it must not go

Fences come from three places: prohibitions written in the instruction files ('never put code at the repo root'), directories the scan classifies as generated or as bulk data, and a built-in list no repository should have to state. A chosen path that violates one is not a warning at the end — it becomes a high-severity risk on the plan.

knowledge/placement.py :: PlacementAdvisor.forbidden_zones

```
    def forbidden_zones(self, candidate: str = "") -> list[ForbiddenZone]:
        """Where this change must not land, and the rule that says so."""
        zones = [
            ForbiddenZone(path=path, reason=reason, source="built-in")
            for path, reason in ALWAYS_FORBIDDEN
            if path in self.index.directories
            or path in {d.split("/")[0] for d in self.index.directories}
        ]
        zones.extend(
            ForbiddenZone(path=", ".join(p.paths), reason=p.text, source=p.source)
            for p in self.conventions.prohibitions
        )
        for note in self.index.directories.values():
            if note.kind == "generated" and not any(z.path == note.path for z in zones):
                zones.append(
                    ForbiddenZone(
                        path=note.path, reason="generated output, not source", source="repo scan"
                    )
                )
        zones.extend(self._bulk_zones())
        if candidate:
            violated = self.conventions.forbids(candidate)
            for prohibition in violated:
                zones.insert(
                    0,
                    ForbiddenZone(
                        path=candidate,
                        reason=f"chosen path violates: {prohibition.text}",
                        source=prohibition.source,
                    ),
                )
        return _dedupe_zones(zones)[:8]
```

10.6 The decision, with its alternatives

Placement is rarely a single defensible answer, so the decision carries the ones it rejected and the price of each: extend the closest existing module, add a sibling beside it, or start a new package. Evidence is ranked — a written rule beats a similarity match, which beats the toolchain default.

`knowledge/placement.py :: PlacementAdvisor._options`

```
        def _options(self, text: str, reuse: list[RepoMatch]) -> list[PlacementOption]:
            options: list[PlacementOption] = []

            rule = self.conventions.best_rule(text)
            if rule is not None:
                options.append(
                    PlacementOption(
                        path=_resolve(rule, text, self._suffix()),
                        approach="new-module",
                        rationale=f"the repo's own rule: '{rule.subject}' → {rule.destination}"
                        f" ({rule.source})",
                        tradeoff="none — this is the documented convention",
                        score=0.9,
                    )
                )

            if reuse:
                # Extending a package's __init__/index file is almost never right:
                # prefer the nearest real module.
                best = next((m for m in reuse if not _is_index(m.path)), reuse[0])
                directory = _parent(best.path)
                options.append(
                    PlacementOption(
                        path=best.path if _is_module(best.path) else directory,
                        approach="extend",
                        rationale=f"{best.path} already covers this area"
                        + (f" ({best.symbol})" if best.symbol else ""),
                        tradeoff="grows an existing file; check it stays cohesive",
                        score=min(0.85, 0.45 + best.score),
                    )
                )
                if directory:
                    options.append(
                        PlacementOption(
                            path=f"{directory}/{_slug(text)}{self._suffix()}",
                            approach="new-module",
                            rationale=f"a sibling of {best.path}, which is the closest existing work",
                            tradeoff="one more file to discover; keeps the existing one small",
                            score=min(0.8, 0.4 + best.score),
                        )
                    )

            default_dir = self._default_dir()
            options.append(
                PlacementOption(
                    path=f"{default_dir}/{_slug(text)}{self._suffix()}" if default_dir else _slug(text),
            # … (see the source file for the rest)
```

10.7 What the pipeline does with it

Stage	Effect
PM	requirements that echo existing code become `spec.context_notes` — a duplicate warning before the plan
Architect	each requirement gets a `PlacementDecision`; components get a `target_path`; fence violations and duplicates become plan risks
Planner	code and test tasks carry their real file paths, and each task description names where it goes, what to read first, and what to avoid
Worker	a backend receives a task that already knows its target file

```
$ python scripts/spec_kit.py where --what 'a new agent type that reviews pull requests'

  goes in   packages/future_agents/agents/…_agent.py   [new-module, confidence 0.9]
  because   the repo's own rule: 'A new agent type' → …/agents/<name>_agent.py (AGENTS.md)
```

```
tests     tests/test_agent_type_reviews.py

read first:
  - packages/future_agents/agents/pdf_agent.py::PDFAgent.agent_type

other approaches:
  - …/agents/pdf_agent.py [extend]  trade-off: grows an existing file
  - packages/…_type_reviews.py [new-module]  trade-off: one more file to discover

must not go in:
  x <root> - Never put code at the repo root. Root is for manifests [AGENTS.md]
  x data/agents/ - bulk data (10000 files across 53 directories) [repo scan]
```

10.8 Embedding it elsewhere

The package is standalone: point it at any repository, in any language, and it works from that repository's own files and instructions.

```
from future_agents.sdd import RepoKnowledge

knowledge = RepoKnowledge.build('../some-other-repo')
knowledge.context('rate limiting on the public API')   # what already exists
decision = knowledge.advise('add per-tenant rate limits')
decision.target_path, decision.test_path, decision.forbidden, decision.alternatives
```

11 · Task graph and execution

11.1 How the graph is built

Unit	Kind	Depends on	Why it exists
Scaffold <component>	code	—	one unit per component, so work can start in parallel
Test REQ-n	test	the component scaffold	the criterion is expressed as a test before code exists
Implement REQ-n	code	its test task	test-first enforced by graph shape, not by discipline
Create missing structure	infra	—	only when the repo lacks required entries
Guardrails and constitution review	review	every code task	the standing gate before delivery
Persona gates	review	the guardrails review	security, migration, eval, perf, observability, ADR
Document the delivered behaviour	doc	review + gates	docs and runbook ship with the change

stages/planner.py :: TaskPlanner.build

```
        def build(self, plan: Plan, spec: Spec) -> TaskGraph:  # noqa: C901
            tasks: list[TaskUnit] = []
            counter = 0

            def next_id() -> str:
                nonlocal counter
                counter += 1
                return f"T-{counter:03d}"

            code_ids: list[str] = []
            for component in plan.components:
                scaffold_id = next_id()
                tasks.append(
                    TaskUnit(
                        id=scaffold_id,
                        title=f"Scaffold {component.name}",
                        description=component.responsibility,
                        kind=TaskKind.CODE,
                        requirement_ids=list(component.requirement_ids),
                        component=component.name,
                        artifacts=[component.target_path] if component.target_path else [],
                        engine=self.router.decide("worker_agent", component.name).engine,
                    )
                )
                code_ids.append(scaffold_id)

                for req_id in component.requirement_ids:
                    requirement = spec.requirement(req_id)
                    if requirement is None:
                        continue
                    criterion_ids = [ac.id for ac in requirement.acceptance_criteria]
                    placement = plan.placement_for(req_id)
                    test_id = next_id()
                    tasks.append(
                        TaskUnit(
                            id=test_id,
                            title=f"Test {req_id}: {_short(requirement.statement)}",
                            description="\n".join(
                                [ac.render() for ac in requirement.acceptance_criteria]
                                + ([f"Run: {self.toolchain.test}"] if self.toolchain else [])
                            ),
                            kind=TaskKind.TEST,
                            requirement_ids=[req_id],
                            criterion_ids=criterion_ids,
                            depends_on=[scaffold_id],
                            component=component.name,
        # … (see the source file for the rest)
```

11.2 Execution order and failure propagation

Tasks run in topological order. A failing task marks its dependents BLOCKED rather than failing the whole run — the rest of the graph still produces useful work, and QA reports precisely which criteria went unverified as a result. A backend that raises is a task failure, never a pipeline crash.

stages/worker.py :: WorkerStage.execute

```
        def execute(self, graph: TaskGraph, spec: Spec) -> list[WorkResult]:
            results: list[WorkResult] = []
            failed: set[str] = set()
            for task in graph.topological_order():
                if failed.intersection(task.depends_on):
                    task.status = TaskStatus.BLOCKED
                    results.append(
                        WorkResult(
                            task_id=task.id,
                            status=TaskStatus.BLOCKED,
                            summary="upstream task failed",
                        )
                    )
                    failed.add(task.id)
                    continue

                task.status = TaskStatus.RUNNING
                started = time.perf_counter()
                try:
                    result = self.backend(task, spec)
                except Exception as exc:  # a backend crash is a task failure, not a run crash
                    result = WorkResult(task_id=task.id, status=TaskStatus.FAILED, error=str(exc)[:500])
                result.duration_ms = round((time.perf_counter() - started) * 1000, 3)
                task.status = result.status
                if result.status in (TaskStatus.FAILED, TaskStatus.BLOCKED):
                    failed.add(task.id)
                results.append(result)
            return results
```

11.3 Backends

The default backend records what would happen. A real backend is one function: shell out to a coding agent, run the repo's own test command, open a pull request. The criterion ids it claims are what QA verifies against, so a backend must claim only what it actually exercised.

stages/worker.py :: dry_run_backend

```
        def dry_run_backend(task: TaskUnit, spec: Spec) -> WorkResult:
            """Default backend: records what *would* happen. Real backends shell out."""
            return WorkResult(
                task_id=task.id,
                status=TaskStatus.DONE,
                summary=f"[dry-run] {task.title}",
                engine=task.engine,
                criterion_ids=list(task.criterion_ids) if task.kind is TaskKind.TEST else [],
                tests_added=[f"test_{task.id.lower().replace('-', '_')}"]
                if task.kind is TaskKind.TEST
                else [],
            )

        import subprocess
        from future_agents.sdd import TaskStatus, WorkResult

        def shell_backend(task, spec):
            # the repo's own command, from the detected toolchain
            command = {
                "test": ["go", "test", "./..."],
                "code": ["make", "build"],
            }.get(task.kind.value)
            if command is None:
                return WorkResult(task_id=task.id, status=TaskStatus.SKIPPED)

            result = subprocess.run(command, capture_output=True, text=True, timeout=900)
            ok = result.returncode == 0
            return WorkResult(
                task_id=task.id,
                status=TaskStatus.DONE if ok else TaskStatus.FAILED,
```

```
            summary=task.title,
            # claim coverage only for a test task that actually passed
            criterion_ids=task.criterion_ids if ok and task.kind.value == "test" else [],
            log_excerpt=result.stdout[-2000:],
            error="" if ok else result.stderr[-2000:],
        )

    pipeline = DeliveryPipeline(config, backend=shell_backend, repo_root='.')
```

The honest limit
Delivery is only as real as the backend wired behind it. Everything upstream — the spec, the plan, the graph, the gates, the QA arithmetic — is real regardless; the dry-run backend simply does no work, and says so in every result it returns.

12 · QA orchestration

QA is not a log reader. It builds a behaviour check for every in-scope acceptance criterion, decides verification from evidence in the work results, computes coverage over MUST criteria, and reports in a fixed, short format.

12.1 BDD and AAA scaffolding

```
criterion  REQ-002-AC-001
  Given    a week of usage data
  When     the churn report runs
  Then     at-risk accounts are listed

test skeleton
  Arrange: a week of usage data
  Act:     the churn report runs
  Assert:  at-risk accounts are listed
```

12.2 Verification rule

```
verified(criterion) =
      ∃ test task covering it whose WorkResult.status == DONE
  AND ∀ code tasks covering it: WorkResult.status == DONE

coverage = |verified MUST criteria| / |MUST criteria|

verdict = BLOCKED  if there are no checks at all
        = FAIL     if any blocker finding, or coverage < required_coverage
        = PASS     otherwise
```

stages/qa.py :: QAStage.verify

```
def verify(self, spec: Spec, graph: TaskGraph, results: list[WorkResult]) -> QAReport:
    qa_cfg = self.config.qa
    by_task = {r.task_id: r for r in results}
    report = QAReport(
        spec_id=spec.id, environment="ephemeral" if qa_cfg.ephemeral_environment else "shared"
    )

    fenced: list[str] = []
    for requirement in spec.requirements:
        for criterion in requirement.acceptance_criteria:
            if self._out_of_scope(requirement.statement, criterion):
                fenced.append(criterion.id)
                continue
            covered_by = [
                t.id
                for t in graph.tasks
                if criterion.id in t.criterion_ids
                and t.kind is TaskKind.TEST
                and by_task.get(t.id)
                and by_task[t.id].status is TaskStatus.DONE
            ]
            impl_ok = all(
                by_task[t.id].status is TaskStatus.DONE
                for t in graph.tasks
                if criterion.id in t.criterion_ids
                and t.kind is TaskKind.CODE
                and t.id in by_task
            )
            check = BehaviourCheck(
                criterion_id=criterion.id,
                requirement_id=requirement.id,
                given=criterion.given,
                when=criterion.when,
                then=criterion.then,
                arrange=f"Arrange: {criterion.given}",
                act=f"Act: {criterion.when}",
                covered_by=covered_by,
                verified=bool(covered_by) and impl_ok,
            )
            check.assert_ = f"Assert: {criterion.then}"
            report.checks.append(check)
```

```
                if not check.verified:
                    report.findings.append(
                        QAFinding(
                            criterion_id=criterion.id,
    # … (see the source file for the rest)
```

12.3 Scope fences

Criteria matching a configured fence are dropped before checks are built and listed in `out_of_scope_ignored`. They can never become findings, so the QA agent cannot halt a pipeline over load testing that was never in scope — the failure mode that makes teams turn automated QA off.

`stages/qa.py :: QAStage._out_of_scope`

```
        def _out_of_scope(self, statement: str, criterion: AcceptanceCriterion) -> bool:
            blob = f"{statement} {criterion.render()}".lower()
            return any(fence.lower() in blob for fence in self.config.qa.out_of_scope)
```

12.4 The reporting protocol

Verbosity is `summary_only` by default: a verdict line, the verified behaviours, then the first blocker. Logs stay in the artifact, out of the channel, unless someone asks for them.

`models.py :: QAReport.summary_lines`

```
        def summary_lines(self) -> list[str]:
            """The summary_only wire format — verbose logs stay out of the channel."""
            verified = [c for c in self.checks if c.verified]
            headline = f"{len(verified)}/{len(self.checks)} behaviours verified"
            lines = [f"QA {self.verdict.value.upper()} — {headline}"]
            blockers = [f for f in self.findings if f.severity == "blocker"]
            if blockers:
                lines.append(f"Blocker: {blockers[0].summary}")
                return lines
            lines.extend(f"✓ {c.then}" for c in verified[:5])
            lines.extend(f"✗ {f.summary}" for f in self.findings[:3])
            return lines
```

```
        QA PASS — 3/3 behaviours verified
        ✓ account managers can call at-risk customers
        ✓ the report pulls from Snowflake every Monday 09:00
        ✓ flag any account whose usage dropped 20% or more
```

12.5 Ephemeral environments

When `qa.ephemeral_environment` is set, the report records the environment as ephemeral and marks it cleaned when the verdict is written — the teardown is part of the protocol, not an afterthought a human remembers.

13 · The memory hub

Agents that forget repeat the same mistake every sprint. After every run the harvester compresses what happened into a case: the objective, the problem, the solution, and — the part that earns its keep — the pitfalls. Cases are markdown on disk, so they are reviewable, diffable and greppable, with a JSON index for retrieval.

13.1 Where pitfalls come from

Source	Becomes
a blocking question that had to be asked	'Intent gap — X had to be asked; answer: Y'
a clarification meeting	'Needed a live meeting: <reason>'
an in-scope QA finding	'QA blocker: <summary>'
a failed task	'Task T-00n failed: <error>'

`memory_hub.py :: _pitfalls`

```
        def _pitfalls(
            clarification: Optional[ClarificationResult],
            qa: Optional[QAReport],
            state: RunState,
        ) -> list[str]:
            out: list[str] = []
            if clarification:
                for question in clarification.questions:
                    if question.blocking and question.answered:
                        out.append(
                            f"Intent gap — '{question.text}' had to be asked; "
                            f"answer: {_first_line(question.answer or '')}"
                        )
                if clarification.meeting:
                    out.append(f"Needed a live meeting: {clarification.meeting.reason}")
            if qa:
                out.extend(f"QA {f.severity}: {f.summary}" for f in qa.findings if f.in_scope)
            out.extend(
                f"Task {w.task_id} failed: {_first_line(w.error)}" for w in state.work_results if w.error
            )
            return out[:12]
```

13.2 Retrieval, biased toward failure

Matching is keyword overlap (Jaccard) with a 1.5× boost for cases whose outcome was not a success. A case that records a pitfall changes the next plan; a success case rarely does. The top-k matches are injected into the plan as `historical_warnings` and as risks with `source=memory`.

`memory_hub.py :: MemoryHub.retrieve`

```
        def retrieve(self, query: str, top_k: Optional[int] = None) -> RetrievalReport:
            """Top-k cases by keyword overlap, with failures weighted up."""
            if not self.config.enabled or not self._cases:
                return RetrievalReport(query=query)
            limit = top_k or self.config.retrieval.max_context_injection
            q_words = _tokens(query)
            matches: list[CaseMatch] = []
            for case in self._cases.values():
                c_words = _tokens(
                    " ".join([case.title, case.objective, case.problem, " ".join(case.tags)])
                )
                if not q_words or not c_words:
                    continue
                score = len(q_words & c_words) / len(q_words | c_words)
                if score < self.config.retrieval.min_score:
                    continue
                reason = "keyword overlap"
                if self.config.retrieval.prefer_failures and case.outcome != "success":
                    score *= 1.5
```

```
                reason = f"{reason}; prior {case.outcome}"
            matches.append(CaseMatch(case=case, score=round(min(score, 1.0), 3), reason=reason))
        matches.sort(key=lambda m: m.score, reverse=True)
        return RetrievalReport(query=query, matches=matches[:limit])
```

13.3 Harvest

`memory_hub.py :: MemoryHub.harvest`

```
    def harvest(self, state: RunState) -> MemoryCase:
        """Synthesise one run into a case. Failures carry the useful lessons."""
        spec = state.spec
        qa = state.qa
        pitfalls = _pitfalls(state.clarification, qa, state)
        failed_work = [w for w in state.work_results if w.error]
        outcome = "success"
        if qa is None or qa.verdict is QAVerdict.BLOCKED:
            outcome = "partial"
        elif qa.verdict is QAVerdict.FAIL or failed_work:
            outcome = "failure"

        solution_bits = []
        if state.plan:
            solution_bits.append(state.plan.architecture)
            solution_bits.extend(f"{c.name}: {c.responsibility}" for c in state.plan.components)
        case = MemoryCase(
            title=spec.title if spec else _first_line(state.objective.statement),
            objective=state.objective.statement,
            problem=(spec.summary if spec else state.objective.context) or "—",
            solution="\n".join(b for b in solution_bits if b) or "—",
            pitfalls=pitfalls,
            tags=_tags(state),
            outcome=outcome,
            requirement_ids=[r.id for r in spec.requirements] if spec else [],
        )
        return self.store(case)
```

13.4 The case format

```
# Weekly churn report for sales

- **Outcome:** failure
- **Tags:** meeting_transcript, reporting, qa-fail
- **Recorded:** 2026-09-05

## Objective
Sales must get a weekly churn report so that account managers can call at-risk customers

## Problem
3 requirements derived from a meeting_transcript submitted by dana.

## Solution
2 components: core (1 req), reporting (2 req). Runtime: Python 3.11+

## Pitfalls & hard lessons
- Intent gap — 'Which system of record supplies this data?' had to be asked;
  answer: Snowflake, refreshed nightly at 02:00
- QA blocker: REQ-003-AC-001 not verified — no passing test task
```

13.5 Swapping in a vector store

Retrieval is deliberately behind one method. A semantic store (Chroma, pgvector, a hosted index) replaces `MemoryHub.retrieve` without any other module noticing; the markdown cases remain the durable, reviewable record.

14 · Engine routing and MCP

The pipeline never names a model inline. It asks the router, which resolves role → engine from the rulebook, lets an intent keyword override it, and falls back when an engine is unavailable. Changing vendor or model is a configuration edit, and a failing engine degrades to deterministic behaviour instead of taking the run down.

14.1 Current role map

Role	Engine	Fallback	Purpose
pm_agent	claude-sonnet-5	claude-opus-5	Turn transcripts and tickets into a traceable spec.md
architect_agent	claude-opus-5	claude-opus-5	Draft plan.md with past pitfalls injected as constraints
planner_agent	claude-sonnet-5	claude-opus-5	Expand the plan into a test-first task DAG
worker_agent	claude-sonnet-5	claude-opus-5	Execute tasks.md and open pull requests
qa_agent	claude-sonnet-5	claude-opus-5	Provision, verify against acceptance criteria, tear down, report
reviewer_agent	claude-opus-5	claude-opus-5	Guardrails and constitution review before delivery
doc_agent	claude-haiku-4-5-20251001	claude-opus-5	Update docs and the delivery record
harvester_agent	claude-haiku-4-5-20251001	claude-opus-5	Compress a finished run into a memory case

Read live from the rulebook. Intent routes: pipeline → claude-opus-5, terraform → claude-opus-5, migration → claude-opus-5

14.2 Resolution order

```
1. intent keyword match ....... 'terraform' in the task intent → claude-opus-5
2. role default ............... agents.roles[role].engine
3. role fallback .............. agents.roles[role].fallback
4. agents.default_engine
5. NullEngine ................. deterministic; the stage's own rules stand
```

`router.py :: EngineRouter.run`

```
def run(self, call: EngineCall, intent: str = "") -> str:
    """Best-effort completion. An engine failure degrades, never crashes."""
    engine, decision = self.engine_for(call.role, intent)
    try:
        out = engine.complete(call)
        self.history.append(RouteRecord(call.role, engine.name, ok=True))
        return out
    except Exception as exc:  # engines are remote; the pipeline must survive
        self.history.append(
            RouteRecord(call.role, decision.engine, ok=False, detail=str(exc)[:200])
        )
        return ""
```

14.3 Engines

An engine is anything with a `name` and a `complete(call)`. Three ship: NullEngine (the default, returns nothing), CallableEngine (any function — the seam tests and custom backends use), and AnthropicEngine (optional `ai` extra).

`router.py :: AnthropicEngine`

```
class AnthropicEngine:
```

```
    """Claude-backed engine. Optional dependency — install the `ai` extra."""

    def __init__(self, model: str = "claude-opus-5", api_key_env: str = "ANTHROPIC_API_KEY"):
        try:
            import anthropic
        except ImportError as exc:  # pragma: no cover - depends on optional extra
            raise RuntimeError("anthropic SDK not installed — pip install -e '.[ai]'") from exc
        key = os.environ.get(api_key_env)
        if not key:
            raise RuntimeError(f"{api_key_env} is not set")
        self.name = model
        self._client = anthropic.Anthropic(api_key=key)

    def complete(self, call: EngineCall) -> str:  # pragma: no cover - network
        response = self._client.messages.create(
            model=self.name,
            max_tokens=call.max_tokens,
            system=call.system,
            messages=[{"role": "user", "content": call.prompt}],
        )
        return "".join(block.text for block in response.content if block.type == "text")
```

Model identifiers
The rulebook uses the current Claude family — claude-opus-5 for architecture and review, claude-sonnet-5 for PM/worker/QA, claude-haiku-4-5-20251001 for documentation and harvesting. Pin the id and record it with the output: a silently upgraded model is an unversioned dependency, which is exactly what the principal AI persona's heuristics say.

14.4 MCP exposure

The gateway URI lives in the rulebook, and the resources an agent needs are already addressable: the constitution as markdown, the golden CI template, the language matrix, the persona catalog and the memory cases. The API surfaces each of these as an endpoint, so an MCP server is a thin adapter rather than a second source of truth.

Resource	Endpoint	CLI
constitution	`GET /api/sdd/constitution`	`spec_kit.py constitution`
golden CI template	`config: cicd.golden_template_path`	`spec_kit.py diff-gate`
language matrix	`GET /api/sdd/languages`	`spec_kit.py languages`
persona catalog	`GET /api/sdd/personas`	`spec_kit.py personas`
memory cases	`GET /api/sdd/cases`	`spec_kit.py cases`

15 · The master orchestrator

Real work rarely lands in one repository: an API change needs a client change needs a pipeline change. The master orchestrator profiles every registered repository, routes an objective to the ones it actually touches, orders them into waves by their declared dependencies, and runs a full delivery pipeline in each — each with its own language, its own toolchain and, if you want, its own persona.

15.1 The part that matters to a human

One question set for the whole program
Five repositories each raise 'which system of record supplies this data?'. The orchestrator merges questions by text, keeps one, and remembers which repo asked what. The human answers once — or attends one meeting — and every affected repo resumes. Without this, multi-repo automation is a machine for generating duplicate questions.

master.py :: MasterOrchestrator._merge_questions

```
        def _merge_questions(self, program: ProgramRun) -> None:
            """One question per distinct unknown, whichever repos raised it."""
            merged: dict[str, Question] = {}
            mapping: dict[str, list[str]] = {}
            meeting: Optional[MeetingRequest] = None

            for repo, state in program.runs.items():
                for question in state.pending_questions():
                    key = question.text.strip().lower()
                    if key not in merged:
                        merged[key] = question.model_copy(deep=True)
                        mapping[merged[key].id] = []
                    merged_id = merged[key].id
                    merged[key].blocking = merged[key].blocking or question.blocking
                    mapping[merged_id].append(f"{repo}:{question.id}")
                clarification = state.clarification
                if (
                    meeting is None
                    and clarification
                    and clarification.outcome is ClarificationOutcome.MEETING_REQUIRED
                    and clarification.meeting
                ):
                    meeting = clarification.meeting.model_copy(deep=True)

            program.questions = list(merged.values())
            program.question_map = mapping
            if meeting and program.questions:
                meeting.title = f"Clarify program: {program.objective.statement[:48]}"
                meeting.agenda = [q.text for q in program.questions]
                meeting.question_ids = [q.id for q in program.questions]
                program.meeting = meeting
            else:
                program.meeting = None
```

15.2 Registration and inventory

master.py :: MasterOrchestrator.register

```
        def register(
            self,
            name: str,
            path: str | Path,
            persona_id: Optional[str] = None,
            keywords: Optional[Iterable[str]] = None,
            depends_on: Optional[Iterable[str]] = None,
        ) -> RepoTarget:
            profile = detect_repo(path)
            target = RepoTarget(
                name=name,
                path=str(path),
                profile=profile,
                persona_id=persona_id or self.persona.id,
                keywords=list(keywords or []),
                depends_on=list(depends_on or []),
```

```
            )
            self.targets[name] = target
            self._pipelines[name] = DeliveryPipeline(
                self.config,
                memory=self.memory,
                backend=self.backend,
                persona=get_persona(target.persona_id),
                repo_root=target.path,
                profile=profile,
            )
            return target

        orchestrator.register('checkout-api',   '../checkout-api',
                              keywords=['api', 'checkout', 'payment'])
        orchestrator.register('web-app',        '../web-app',
                              keywords=['ui', 'web', 'checkout'],
                              depends_on=['checkout-api'])
        orchestrator.register('platform-infra', '../platform-infra',
                              persona_id='staff_platform',
                              keywords=['infra', 'deploy'])

        checkout-api    go          missing: none
        web-app         typescript  missing: none
        platform-infra  terraform   missing: ['docs/runbook.md']
```

15.3 Routing and waves

An explicit repo list always wins. Otherwise the objective is scored against each repo's name, keywords and language; if nothing matches, every repo is in scope rather than silently none. Waves come from the declared dependency edges, and a cycle raises instead of deadlocking.

`master.py :: MasterOrchestrator.waves`

```
        def waves(self, targets: list[RepoTarget]) -> list[list[str]]:
            """Order repos by their declared dependencies; raises on a cycle."""
            names = {t.name for t in targets}
            pending = {t.name: {d for d in t.depends_on if d in names} for t in targets}
            waves: list[list[str]] = []
            while pending:
                ready = sorted(name for name, deps in pending.items() if not deps)
                if not ready:
                    raise CycleError(f"cycle in repo dependencies: {', '.join(sorted(pending))}")
                waves.append(ready)
                for name in ready:
                    del pending[name]
                for deps in pending.values():
                    deps.difference_update(ready)
            return waves
```

15.4 Dependency behaviour

A repo whose dependency is still clarifying or blocked is skipped with the reason recorded, and picked up automatically once the dependency reaches a usable state — so a program converges over successive answer rounds rather than needing to be restarted.

`master.py :: MasterOrchestrator._resume_blocked_waves`

```
        def _resume_blocked_waves(self, program: ProgramRun) -> None:
            """Repos skipped behind a dependency get their turn once it clears."""
            for wave in program.waves:
                for name in wave:
                    if name in program.runs or name not in program.skipped:
                        continue
                    target = self.targets[name]
                    if self._unmet_dependency(program, target):
                        continue
                    del program.skipped[name]
                    state = self._pipelines[name].start(self._repo_objective(program.objective, target))
                    program.runs[name] = state
                    program.events.append(
                        ProgramEvent(repo=name, message=f"resumed — stage {state.stage.value}")
                    )
```

15.5 Per-repo context

Each repository receives its own copy of the objective, carrying that repo's language, test command and dependency policy as constraints — which is why the Go repo's plan says `go test ./...` and the TypeScript repo's says `npm test` from the same human sentence.

master.py :: MasterOrchestrator._repo_objective

```
        def _repo_objective(self, objective: Objective, target: RepoTarget) -> Objective:
            """A per-repo copy carrying that repo's toolchain as context."""
            chain = target.profile.toolchain()
            copy = objective.model_copy(deep=True)
            copy.id = f"{objective.id}-{target.name}"
            copy.metadata = {
                **objective.metadata,
                "repo": target.name,
                "language": chain.language,
                "test_command": chain.test,
            }
            copy.constraints = [
                *objective.constraints,
                f"Repository {target.name} is {chain.display_name}; tests run with "
                f"`{chain.test or 'no configured test command'}`.",
                f"Dependency ranges: {chain.pin_rule}" if chain.pin_rule else "",
            ]
            copy.constraints = [c for c in copy.constraints if c]
            return copy
```

15.6 A program run

```
        $ python scripts/spec_kit.py program \
            --repo checkout-api=../checkout-api \
            --repo web-app=../web-app \
            --depends web-app:checkout-api \
            --source meeting_transcript --by dana --input notes.txt \
            --statement 'Add saved payment methods to checkout'

        program prog-5a75e8f6 — waves: [['checkout-api'], ['web-app']]
          checkout-api           clarify
          web-app                skipped — waiting on checkout-api (clarify)

        open questions (answer once for the whole program):
          ! q-1c9f  Which external system is on the other side, and who owns its credentials?
          ! q-77a2  This touches payment — who signs off before it ships?

        # after two answer rounds
          checkout-api  done  3 requirements  13 tasks  QA pass  accepted
          web-app       done  3 requirements  13 tasks  QA pass  accepted
```

15.7 The program report

master.py :: ProgramRun.report

```
        def report(self) -> dict[str, object]:
            return {
                "program": self.id,
                "objective": self.objective.statement,
                "waves": self.waves,
                "repos": {
                    repo: {
                        "stage": state.stage.value,
                        "requirements": len(state.spec.requirements) if state.spec else 0,
                        "tasks": len(state.tasks.tasks) if state.tasks else 0,
                        "qa": state.qa.verdict.value if state.qa else None,
                        "accepted": state.delivery.accepted if state.delivery else False,
                        "case": state.case_id,
                    }
                    for repo, state in self.runs.items()
                },
                "skipped": self.skipped,
                "open_questions": [q.text for q in self.questions],
                "meeting": self.meeting.title if self.meeting else None,
                "blockers": self.blockers(),
            }
```

16 · Configuration reference

One rulebook, loaded by every surface — pipeline, CLI, API and CI. ${VAR} and ${VAR:-default} resolve from the environment at load time, and a literal value under a key that looks like a secret is rejected with a ConfigError rather than being caught later by a scanner.

16.1 Every key

Section	Key	Meaning
project	name / description / owner	identity for reports and cases
governance	runtime_stack	the default stack when no repo is attached
governance	banned_practices	matched against plans; a hit fails the plan gate
governance	escalation_triggers	words that force a named human approver
governance	stack_terms	implementation words a functional spec must avoid
governance	enforce_test_parity / spec_purity	toggle the two structural gates
agents	mcp_gateway_uri	where the MCP gateway lives
agents	default_engine	used when a role names none
agents	roles.<role>.engine / fallback	per-role model, with a fallback
agents	intent_routes	keyword → engine, checked before the role default
memory_hub	case_studies_path	where markdown cases are written
memory_hub	retrieval.max_context_injection	how many past cases enter a plan
memory_hub	retrieval.prefer_failures	weight failures above successes
clarification	ready_threshold	confidence at which the system starts building
clarification	meeting_threshold	below this, a meeting instead of a form
clarification	max_rounds	async rounds before escalating
clarification	auto_assume_low_risk	turn low-risk unknowns into assumptions
clarification	meeting_attendees / duration	who is invited, for how long
cicd	enforce_golden_pattern	run the diff gate
cicd	golden_template_path	the approved pipeline shape
qa	enforce_bdd / enforce_aaa	Given-When-Then and Arrange-Act-Assert
qa	out_of_scope	fences QA may never fail a run over
qa	required_coverage	fraction of MUST criteria that must verify
qa	communication.verbosity	summary_only keeps logs out of the channel

16.2 Secret handling

config.py :: _resolve

```
        def _resolve(node: Any, trail: str) -> Any:
            """Recursively expand `${VAR}` and refuse literal secrets."""
            if isinstance(node, dict):
                return {k: _resolve(v, f"{trail}.{k}" if trail else str(k)) for k, v in node.items()}
            if isinstance(node, list):
                return [_resolve(v, f"{trail}[{i}]") for i, v in enumerate(node)]
            if isinstance(node, str):
                match = _ENV_REF.match(node.strip())
```

```
            if match:
                name, default = match.group(1), match.group(2)
                value = os.environ.get(name, default)
                if value is None:
                    raise ConfigError(f"{trail}: environment variable {name} is not set")
                return value
            leaf = trail.rsplit(".", 1)[-1]
            if _SECRET_KEY.search(leaf) and node.strip():
                raise ConfigError(
                    f"{trail}: inline secret — use ${{ENV_VAR}} or a secrets manager instead"
                )
        return node
```

16.3 The rulebook in full

data/config/spec_kit/spec-kit-enterprise.yaml

```
# Spec Kit — the single rulebook for automated delivery.
# Loaded by future_agents.sdd.SpecKitConfig.load(); exposed to agents over MCP.
# Values may reference the environment as ${VAR} or ${VAR:-default}.
# Never write a literal credential here — the loader rejects it.

version: "1.0"

project:
  name: "enterprise-automation-core"
  description: "Objective in, verified delivery out, lesson recorded."
  owner: "platform-engineering"

governance:
  runtime_stack: "Python 3.11+, FastAPI, Pydantic v2"
  banned_practices:
    - "No direct database connections from API route handlers."
    - "No secrets in code — environment variables or a secrets manager only."
    - "No exact version pins without written justification."
  required_practices:
    - "Every MUST requirement carries at least one test task."
    - "Every acceptance criterion is written Given/When/Then."
    - "Every assumption is recorded and surfaced at delivery."
  security_boundaries:
    - "Agents never hold long-lived production credentials."
    - "Outbound calls carry no user financial or personal context."
  # Free-text matches here force a named human approver before ship.
  escalation_triggers:
    - "auth"
    - "payment"
    - "pii"
    - "phi"
    - "migration"
    - "production credential"
  # Implementation vocabulary that must not appear in a functional spec.
  stack_terms:
    - "postgres"
    - "kubernetes"
    - "react"
    - "kafka"
    - "redis"
    - "lambda"
  enforce_test_parity: true
  enforce_spec_purity: true

agents:
  mcp_gateway_uri: "${SPEC_KIT_MCP_GATEWAY:-mcp://internal-devops-gateway:8080}"
    # … (see the source file for the rest)
```

17 · Operating the system

17.1 Command line

Command	What it does
`run --statement … [--input f] [--repo path]`	intake an objective and drive it as far as it can go
`answer --state f --answer ID=text`	answer open questions and resume
`meeting --state f --notes-file f`	record a clarification meeting and resume
`status --state f`	print a saved run
`program --repo name=path --statement …`	one objective across many repositories
`detect --path .`	profile a repo: language, toolchain, structure gaps
`scaffold --path . [--language X] --write`	create the structure a repo is missing
`cases [--query …]`	browse or search the memory hub
`personas / languages`	list seniority profiles and toolchains
`constitution`	render governance as markdown (MCP resource)
`diff-gate --proposed f`	check a pipeline change against the golden template

17.2 HTTP

Served by `uvicorn future_agents.api.main:app`.

Method / path	Purpose
`POST /api/sdd/objectives`	intake — wire a meeting-notes webhook here
`GET /api/sdd/runs/{id}/questions`	what the system needs from a human
`POST /api/sdd/runs/{id}/answers`	answer and resume
`POST /api/sdd/runs/{id}/meeting`	record a meeting and resume
`POST /api/sdd/programs`	run one objective across many repositories
`POST /api/sdd/programs/{id}/answers`	one answer sheet for the whole program
`POST /api/sdd/repos/detect`	profile a repository
`POST /api/sdd/repos/scaffold`	plan or write missing structure
`GET /api/sdd/cases`	search the memory hub
`GET /api/sdd/personas · /languages`	catalogs
`GET /api/sdd/constitution`	governance as markdown
`POST /api/sdd/cicd/diff-gate`	golden-pattern check

17.3 Python

```
from future_agents.sdd import (
    DeliveryPipeline, MasterOrchestrator, Objective, SpecKitConfig, get_persona,
)

config = SpecKitConfig.load()
pipeline = DeliveryPipeline(
    config,
    persona=get_persona('principal_hybrid'),
    repo_root='.',              # tasks use this repo's toolchain and structure
```

```
    backend=shell_backend,      # real work; omit for a dry run
)

state = pipeline.start(Objective(
    statement='Sales must get a weekly churn report so that AMs can call at-risk accounts',
    source=IntakeSource.MEETING,
    submitted_by='dana',
    raw_inputs=[transcript_text],
    constraints=['no new production env vars'],
    deadline='2026-10-01',
))

if state.awaiting_human:
    for question in state.pending_questions():
        print(('!' if question.blocking else '·'), question.id, question.text)
    state = pipeline.answer(state, collect_answers())

print(state.qa.summary_lines())
print(state.delivery.accepted, state.delivery.unconfirmed_assumptions)
save_state(state, '.spec-kit/runs')
```

17.4 In CI

```
# .github/workflows/spec-kit.yml
name: spec-kit

on:
  pull_request:

jobs:
  structure:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python scripts/spec_kit.py detect --path .
      - run: python scripts/spec_kit.py diff-gate --proposed .github/workflows/ci.yml
```

17.5 Intake from a meeting tool

The API is the integration point: post the transcript as `raw_inputs` with `source=meeting_transcript`, then surface the returned questions wherever the team already is. If the response carries a meeting request, put it on a calendar — the agenda is already written.

```
POST /api/sdd/objectives
{
  "statement": "Add saved payment methods to checkout",
  "source": "meeting_transcript",
  "submitted_by": "dana",
  "raw_inputs": ["Dana: the api must expose a tokenised card list …"],
  "constraints": ["no card PAN stored"],
  "deadline": "2026-11-01"
}

200 OK
{
  "run_id": "run-0ddf86a9", "stage": "clarify", "awaiting_human": true,
  "confidence": 0.54, "outcome": "async_questions",
  "questions": [{"id": "q-1c9f", "text": "Which external system …", "blocking": true}],
  "meeting": null
}
```

18 · Pattern catalog

The design patterns this system is built from, each with the failure it prevents and the price it charges. They are reusable outside this codebase — most of them are answers to problems every agentic delivery system meets.

18.1 IR pipeline

Problem	Free-form intent produces inconsistent output and cannot be reviewed.
Solution	Pass intent through typed artifacts, each bounding the next; a stage may only read its immediate input.
In code	`models.py, pipeline.py`
Trade-off	More upfront structure; a genuinely trivial task still walks the whole pipeline.

18.2 Fail-closed gate

Problem	An agent fills a gap with a plausible guess and the error surfaces three stages later.
Solution	A gate returns violations; an error-severity violation stops the stage with the reason recorded on the run.
In code	`constitution.py, pipeline._block`
Trade-off	Runs stop more often — which is the point, but it needs a human in the loop.

18.3 Confidence-scored intent

Problem	'Is this clear enough?' is a judgement call made differently every time.
Solution	Detectors emit weighted signals; confidence is arithmetic; thresholds live in config.
In code	`clarify.IntentClarifier`
Trade-off	Heuristic detectors are approximate; the score is a prior, not a truth.

18.4 Escalation ladder

Problem	Either the agent asks nothing, or it asks everything and becomes exhausting.
Solution	Four rungs — auto-assume, async questions, meeting, blocked — chosen by score and by how many rounds have already failed to resolve it.
In code	`clarify._decide`
Trade-off	Thresholds need tuning per team; too high and every objective becomes a meeting.

18.5 Assumption ledger

Problem	Silent defaults are indistinguishable from decisions.
Solution	Every unknown resolved without a human becomes an Assumption record, surfaced on the delivery.
In code	`models.Assumption, DeliveryStage`
Trade-off	The delivery record grows; that is the honest cost of not asking.

18.6 Traceability ids

Problem	'Is this requirement covered?' can only be answered by reading everything.
Solution	REQ → AC → task → QA check, by id, all the way through.
In code	`models.py, stages.py`
Trade-off	Ids must be stable across re-planning; renumbering breaks history.

18.7 Content-hash staleness

Problem	An upstream artifact is revised and downstream work silently continues on the old one.
Solution	Each artifact fingerprints its semantic content; a downstream artifact stores the hash it was built from and the gate compares.
In code	`models.Hashable, constitution.check_plan`
Trade-off	Any edit invalidates downstream work, including a cosmetic one.

18.8 Test-first by graph shape

Problem	'Write tests' is advice, and advice is skipped under time pressure.
Solution	The implement task depends on its test task; test parity is also a gate on the graph.
In code	`stages.TaskPlanner.build`
Trade-off	Rigid for exploratory spikes — use the pragmatic persona there.

18.9 Scope fence

Problem	An automated QA agent invents requirements and blocks releases over them.
Solution	Configured fences are removed before checks are built, so they cannot become findings.
In code	`stages.QAStage._out_of_scope, qa.out_of_scope`
Trade-off	A real gap inside a fence is invisible; fences must be reviewed like code.

18.10 Summary-only reporting

Problem	Verbose agent output trains humans to ignore it.
Solution	A fixed short format: verdict, verified behaviours, first blocker. Logs live in the artifact.
In code	`models.QAReport.summary_lines`
Trade-off	Debugging needs one extra hop to the full record.

18.11 Case-based memory

Problem	The same mistake is made in three sprints by three people.
Solution	Harvest each run into a markdown case; retrieve the closest before planning; weight failures above successes.
In code	`memory_hub.py`
Trade-off	Keyword retrieval is shallow; cases need occasional pruning.

18.12 Router with fallback

Problem	A model id hard-coded in a stage makes vendor change a refactor, and an outage an incident.
Solution	Role and intent resolve to an engine through config, with a fallback and a deterministic null engine at the end of the chain.
In code	`router.EngineRouter`
Trade-off	Silent degradation must be visible — hence the routing history.

18.13 Deterministic core, model enrichment

Problem	A pipeline whose structure depends on a model cannot be tested or replayed.
Solution	Rules produce the structure; the engine only improves free text; empty output is a valid response.
In code	`stages.py`
Trade-off	Heuristic extraction is blunter than a good model on messy input.

18.14 Persona as configuration

Problem	'Act like a senior engineer' in a prompt changes tone, not behaviour.
Solution	A persona tunes thresholds, adds review tasks and injects heuristics as plan risks.
In code	`personas.py`
Trade-off	Heuristic keyword matching can fire a gate on an unrelated word.

18.15 Toolchain matrix

Problem	Automation that assumes one language fails on the second repository.
Solution	Detect the repo; read every command, layout and dependency policy from a Toolchain entry.
In code	`languages.py`
Trade-off	A new ecosystem needs an entry before the system understands it.

18.16 Idempotent scaffold

Problem	Structure generators overwrite work and cannot be run twice.
Solution	Plan what is missing, write only that, treat conventional alternatives as satisfying the requirement, and never create a forbidden file.
In code	`scaffold.py`
Trade-off	Cannot repair a wrong-but-present file; it only fills gaps.

18.17 Dependency waves

Problem	Multi-repo work fans out in the wrong order and the client ships against a missing API.
Solution	Declared repo dependencies produce ordered waves; a repo whose dependency is unresolved is skipped and resumed later.
In code	`master.waves, master._resume_blocked_waves`
Trade-off	Dependencies are declared, not inferred — a wrong edge is a wrong order.

18.18 Merged question set

Problem	Five repos ask a human the same question five times.
Solution	Questions are merged by text across repos, answered once, and fanned back out by a question map.
In code	`master._merge_questions`
Trade-off	Two repos can mean subtly different things by the same sentence.

18.19 Resumable run state

Problem	A pipeline that pauses for a human dies with the process.
Solution	All state lives in one serialisable model; the pipeline is stateless machinery over it.
In code	`models.RunState, save_state / load_state`
Trade-off	State migrations become a real concern as the schema evolves.

19 · Extending the system

19.1 Add a language

One entry in `TOOLCHAINS` — see §8.5. Nothing else changes.

19.2 Add a persona

```
from future_agents.sdd.personas import Heuristic, Persona, ReviewGate, PERSONAS

EMBEDDED = Persona(
    id="principal_embedded",
    title="Principal Embedded Engineer",
    years_experience=25,
    disciplines=["firmware", "rtos", "hardware-interfaces"],
    ready_threshold=0.85,          # silicon does not forgive a vague spec
    required_coverage=1.0,
    heuristics=[
        Heuristic(
            text="Every ISR has a stated worst-case execution time.",
            applies_to=("interrupt", "isr", "timer", "dma"),
            severity="high",
        ),
    ],
    gates=[
        ReviewGate(
            name="Memory and timing review",
            description="Stack depth, heap use and worst-case timing are bounded.",
        ),
    ],
)
PERSONAS[EMBEDDED.id] = EMBEDDED
```

19.3 Add a clarification detector

A detector is a function from an objective and its context to signals. Register it on the clarifier and it participates in scoring immediately.

```
from future_agents.sdd.clarify import IntentClarifier, Signal
from future_agents.sdd.models import QuestionTopic

def detect_missing_retention(objective, ctx):
    if "store" not in ctx.text and "retain" not in ctx.text:
        return []
    if any(w in ctx.text for w in ("retention", "delete after", "ttl")):
        return []
    return [
        Signal(
            topic=QuestionTopic.DATA,
            question="How long is this data kept, and what deletes it?",
            why="undeclared retention becomes a compliance finding",
            weight=0.18,
            blocking=True,
        )
    ]

clarifier = IntentClarifier(config)
clarifier._detectors.append(detect_missing_retention)
```

19.4 Add a governance rule

Add a method to `Constitution` returning `Violation` objects, and call it from the matching stage transition in `DeliveryPipeline._build`. Error severity stops the run; warn severity is recorded.

19.5 Add a stage

Stages are plain classes with one method that takes upstream artifacts and returns the next one. Add the artifact to `RunState`, add the enum member to `Stage`, and wire it into the machine

between the two stages it belongs between. Keep it deterministic; let the engine enrich only free text.

19.6 Replace the memory backend

Implement `retrieve()` against a vector store and keep the markdown cases as the durable record — see §13.5.

20 · Testing

The whole pipeline runs offline and deterministically, which is what makes it testable at all. Two suites cover it: `tests/test_sdd.py` for the core pipeline and `tests/test_sdd_multirepo.py` for personas, languages, scaffolding and the orchestrator.

Area	What is asserted
Clarification	well-formed intent is ready; vague intent escalates to a meeting; answers raise confidence; low-risk unknowns become assumptions; escalation triggers block
Config	env references resolve; inline secrets are rejected; thresholds must be ordered
Constitution	missing criteria, blocking unknowns, stale plans, banned practices, test parity; the diff gate allows additive patches and blocks rewrites
IR models	content hashes ignore provenance and track content; topological order is stable; cycles raise
Stages	requirement extraction and ids; memory warnings reach the plan; test-before-code; failure blocks dependents; fences; coverage arithmetic
Personas	thresholds and coverage change; gates appear in the graph; AI heuristics fire only on model work; unknown ids fall back
Languages	every toolchain declares the essentials; scaffold → detect round-trips for seven languages; TS beats JS; unknown still profiles
Scaffolding	required structure is created; idempotent; dry-run writes nothing; forbidden files never appear; CI uses the language's commands
Orchestrator	routing, waves, cycle rejection, merged questions, one answer sheet drives every repo, per-repo toolchains

20.1 Running them

```
pytest -q                                    # whole repo
pytest -q tests/test_sdd.py                  # core pipeline
pytest -q tests/test_sdd_multirepo.py        # personas, languages, orchestration
ruff check packages/future_agents/ apps/ scripts/
ruff format --check packages/future_agents/ apps/ scripts/
python packages/guardrails/guardrails_engine.py . --mode block
```

20.2 Testing your own backend

Use `CallableEngine` for the model seam and a fake backend for the work seam; both are single functions, so a full delivery run in a test is fast and has no network.

```
def failing_backend(task, spec):
    if task.id == 'T-001':
        return WorkResult(task_id=task.id, status=TaskStatus.FAILED, error='boom')
    return WorkResult(task_id=task.id, status=TaskStatus.DONE)

results = WorkerStage(failing_backend).execute(graph, spec)
assert any(r.status is TaskStatus.BLOCKED for r in results)
```

21 · Limits, and what to build next

Stated plainly, because a system that oversells itself gets switched off the first time it is believed.

Limit	Consequence	Mitigation today
Requirement extraction is heuristic	a rambling transcript yields noisy requirements	attach an engine to pm_agent; edit the spec before planning
Memory retrieval is keyword-based	a semantically similar case can be missed	tag cases; swap in a vector store behind retrieve()
dry_run_backend does no work	delivery is simulated until a backend is wired	wire a shell/agent backend (§11.3)
Detectors are keyword-driven	an unusual phrasing can slip past a gate	add a detector (§19.3); gates still catch the artifact
API runs live in memory	a restart loses in-flight runs	save_state / load_state; persist per run
Repo dependencies are declared	a wrong edge produces a wrong order	keep the graph small and reviewed
One engine call per free-text field	prose quality varies with the engine	the structure never does — that is the point

21.1 The next things worth building

- **A real worker backend** in this repo: shell out per task kind, run the detected toolchain's test command, and open a pull request per component.
- **Semantic memory**: embeddings behind `retrieve()`, with the markdown cases unchanged as the reviewable record.
- **Persistence for programs**: the multi-repo run state on disk, so a program survives a restart the way a single run already does.
- **Cost and latency accounting** per run, which the AI persona's own heuristics demand of anything it ships.
- **An MCP server** exposing the constitution, golden templates, language matrix and cases directly — the API already serves each of them.

21.2 Where to start reading the code

If you want to understand…	Read
how a run flows	`pipeline.py — DeliveryPipeline._build`
why it stops and asks	`clarify.py — IntentClarifier.assess and _decide`
what the artifacts are	`models.py, top to bottom`
how governance is enforced	`constitution.py`
how seniority changes behaviour	`personas.py`
how any language is supported	`languages.py`
how many repos are coordinated	`master.py`

One sentence
The system refuses to build on a guess, asks a human at the right level of ceremony, ships with tests and gates appropriate to a principal engineer, works in whatever language the repository is written in, coordinates several repositories at once, and writes down what it learned so the next run starts better informed.